ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



Nguyễn Lê Minh

PHÁT HIỆN LỖ HỔNG PHÂN QUYỀN CÓ TRẠNG THÁI TRONG REST API

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



NGUYỄN LÊ MINH

PHÁT HIỆN LỖ HỔNG PHÂN QUYỀN CÓ TRẠNG THÁI TRONG REST API

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: ThS. Trần Mạnh Cường

Cán bộ đồng hướng dẫn: TS. Nguyễn Thị Thủy

HÀ NỘI - 2026

VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY



Nguyen Le Minh

DETECTING STATEFUL AUTHORIZATION VULNERABILITIES IN REST APIs

BACHELOR'S THESIS

Major: Information Technology

Supervisor: MSc. Tran Manh Cuong

Co-Supervisor: Dr. Nguyen Thi Thuy

HANOI - 2026

Lời cảm ơn

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất tới ThS. Trần Mạnh Cường và TS. Nguyễn Thị Thủy, những người đã tận tình chỉ dẫn, định hướng khoa học và động viên tôi trong suốt quá trình nghiên cứu và hoàn thành khóa luận này.

Tôi xin chân thành cảm ơn các thầy cô giáo tại Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội. Các thầy cô đã truyền đạt kiến thức nền tảng quý báu trong suốt những năm tháng học tập tại trường.

Cuối cùng, tôi xin bày tỏ lòng biết ơn sâu sắc tới gia đình và người thân. Gia đình đã luôn ủng hộ, chia sẻ và tạo mọi điều kiện để tôi hoàn thành khóa luận tốt nghiệp.

Hà Nội, tháng 11 năm 2026

Nguyễn Lê Minh

Lời cam đoan

Tôi là Nguyễn Lê Minh, sinh viên lớp K68I-IT3, mã sinh viên 23020114. Tôi xin cam đoan khóa luận tốt nghiệp này là công trình nghiên cứu do chính tôi thực hiện dưới sự hướng dẫn của ThS. Trần Mạnh Cường và TS. Nguyễn Thị Thủy. Mọi kết quả thực nghiệm và số liệu đo trong khóa luận đều trung thực và lấy từ các lượt chạy thực tế. Công trình không sao chép từ bất kỳ tài liệu nào khác mà không được trích dẫn rõ ràng.

Tuyên bố về sự hỗ trợ của AI

Trong quá trình thực hiện khóa luận, tôi sử dụng công cụ trí tuệ nhân tạo Claude (Anthropic) để tóm tắt tài liệu, gợi ý hướng xử lý kỹ thuật và rà soát mã nguồn các dự án mã nguồn mở nhằm tìm ứng dụng có lỗi phù hợp cho thực nghiệm. Nhãn khởi đầu của dữ liệu huấn luyện mô hình Naive Bayes do hai mô hình Claude Sonnet và Claude Opus gán (Mục 3.4). Tôi đã kiểm tra lại các đề xuất của công cụ và chịu trách nhiệm về kết quả cuối cùng của khóa luận.

Hà Nội, ngày ..... tháng ..... năm 2026

Sinh viên

Nguyễn Lê Minh

Tóm tắt

Kiểm soát truy cập giới hạn những thao tác mà mỗi người dùng được phép thực hiện trên REST API. Các kỹ thuật kiểm thử tự động hiện nay chủ yếu tập trung vào lỗi phân quyền phi trạng thái, như lỗi ở mức đối tượng (BOLA) hoặc mức chức năng (BFLA). Lớp lỗ hổng phân quyền có trạng thái (Stateful Broken Access Control) ít được các kỹ thuật này kiểm tra trực tiếp. Lỗi này xuất hiện khi quyền hạn của người dùng bị thu hồi trong vận hành, nhưng hệ thống vẫn chấp nhận các phiên làm việc cũ.

Khóa luận đề xuất giải pháp tự động phát hiện lớp lỗ hổng này bằng kiểm thử biến hình trên công cụ EvoMaster. Giải pháp dựa trên quan hệ MR-A1: nếu một thao tác cần đúng quyền vừa bị thu hồi và đã thành công trước khi tài khoản bị hạ quyền, thì khi gửi lại bằng cùng token sau đó, thao tác phải bị từ chối. Mỗi kịch bản kiểm thử gồm bộ ba thao tác: thao tác gốc, thao tác hạ quyền và thao tác kiểm chứng. Mô hình Naive Bayes tự động nhận diện thao tác hạ quyền từ đặc tả OpenAPI, nên người dùng không phải tự khai báo thao tác này.

Thực nghiệm gồm 98 lượt chạy ở chế độ hộp đen trên bốn ứng dụng mục tiêu. reports-api và vampi-seeded mang lỗi tự gieo; pyLoad mang lỗ hổng CVE-2026-41133; artifact-keeper mang một lỗi mà nhà phát triển đã vá. Trên các biến thể có lỗ hổng, công cụ phát hiện vi phạm ở 19/20 lượt với reports-api, 6/20 lượt với vampi-seeded, 5/5 lượt với pyLoad và 4/5 lượt với artifact-keeper. Trong 48 lượt chạy trên các biến thể đã vá, công cụ dựng được bộ ba ở 30 lượt và không báo động giả ở lượt nào.

Từ khóa: REST API, kiểm soát truy cập, phân quyền có trạng thái, kiểm thử biến hình, EvoMaster

Abstract

Access control restricts the operations that each user is allowed to perform on a REST API. Current automated testing techniques mainly target stateless authorization flaws, such as object-level (BOLA) or function-level (BFLA) flaws. Stateful broken access control is rarely checked directly by these techniques. This flaw occurs when a user's privileges are revoked at runtime, but the system keeps accepting the user's existing sessions.

This thesis proposes an automated approach to detect this class of vulnerabilities through metamorphic testing in the EvoMaster tool. The approach is based on the metamorphic relation MR-A1: if an operation needs the revoked privilege and succeeded before the account was downgraded, the same request sent again later with the same token must be rejected. Each test scenario consists of an operation triple: a source operation, a downgrade transition and a follow-up. A Naive Bayes model automatically identifies downgrade operations from the OpenAPI specification, so users do not have to declare them by hand.

The experiments comprise 98 black-box runs on four target applications. Two of them, reports-api and vampi-seeded, carry seeded bugs. The third, pyLoad, carries the vulnerability CVE-2026-41133, and the fourth, artifact-keeper, carries a bug that its developers have since fixed. On the vulnerable variants, the tool detected violations in 19/20 runs on reports-api, 6/20 runs on vampi-seeded, 5/5 runs on pyLoad and 4/5 runs on artifact-keeper. Across 48 runs on the fixed variants, the tool built a valid triple in 30 runs and raised no false alarm in any of the 48.

Keywords: REST API, access control, stateful authorization, metamorphic testing, EvoMaster

Mục lục

Lời cảm ơn

Lời cam đoan **i**

Tóm tắt **ii**

Abstract **iii**

Mục lục **iv**

Danh sách hình vẽ **vi**

Danh sách bảng **vii**

Danh mục các từ viết tắt **viii**

Chương 1 Giới thiệu **1**

1.1 Bối cảnh và động lực nghiên cứu 1

1.2 Phát biểu bài toán 2

1.3 Mục tiêu và phạm vi nghiên cứu 3

1.4 Đóng góp của khóa luận 3

1.5 Bố cục khóa luận 4

Chương 2 Cơ sở lý thuyết và nghiên cứu liên quan **5**

2.1 Kiểm thử REST API dựa trên tìm kiếm 5

2.2 Kiểm thử biến hình 6

2.3 Lỗ hổng kiểm soát truy cập trong REST API 6

2.4 Các công trình nghiên cứu liên quan 8

Chương 3 Phương pháp đề xuất **9**

3.1 Tổng quan Oracle 970 9

3.2 Quan hệ biến hình MR-A1 9

3.3 Bộ ba thao tác và điều kiện tiền đề 13
3.3.1 Cấu trúc bộ ba thao tác 13
3.3.2 Điều kiện tiền đề và cách chọn source 14
3.3.3 Bộ ba bị hủy và bước đăng nhập lại 14
3.3.4 Thuật toán dựng kịch bản kiểm thử MR-A1 16
3.4 Tự động nhận diện và xếp hạng thao tác chuyển trạng thái 17

Chương 4 Cài đặt thực nghiệm và tích hợp hệ thống 21
4.1 Tích hợp Oracle 970 vào mã nguồn EvoMaster 21
4.2 Ứng dụng mục tiêu 22
4.2.1 Ứng dụng reports-api 23
4.2.2 Ứng dụng vampi-seeded 24
4.2.3 Ứng dụng pyLoad 24
4.2.4 Ứng dụng artifact-keeper 25
4.3 Thiết lập thực nghiệm 25

Chương 5 Thực nghiệm và đánh giá kết quả 27
5.1 Thiết kế thực nghiệm 27
5.2 Hiệu quả phát hiện và độ chính xác 27
5.3 Hiệu quả xếp hạng thao tác chuyển trạng thái 29
5.4 Nghiên cứu tình huống trên lỗi thật 30
5.5 Phân tích nguyên nhân bỏ sót và giới hạn 31
5.6 Đối chứng với baseline và kiểm tra upstream 33

Chương 6 Kết luận và hướng phát triển 34
6.1 Tổng kết kết quả nghiên cứu 34
6.2 Hạn chế hiện tại của đề tài 35
6.3 Hướng phát triển trong tương lai 35

Tài liệu tham khảo 36

Phụ lục A Danh mục endpoint của các ứng dụng mục tiêu 39

Danh sách hình vẽ

1.1 Vòng đời lỗ hổng phân quyền có trạng thái 2

2.1 Chu trình kiểm thử EvoMaster với pha kiểm thử bảo mật 6

3.1 Trình tự thực hiện bộ ba thao tác và kích hoạt Oracle 970 10
3.2 Hai trạng thái phân quyền trong MR-A1 12
3.3 Quy trình nhận diện thao tác hạ quyền từ đặc tả OpenAPI 18

4.1 Các thành phần của phần mở rộng trong pha kiểm thử bảo mật . . . 22
4.2 Trích ca kiểm thử 970 do công cụ sinh trên reports-api (rút gọn) . . 23

Danh sách bảng

3.1 Ánh xạ giá trị quan sát sang mã trạng thái HTTP 11
3.2 Đối chiếu oracle so sánh chéo tài khoản và quan hệ MR-A1 12
3.3 Kịch bản kiểm thử MR-A1 trên reports-api 13
3.4 Luật thử lại và bỏ qua của bộ dựng 18
3.5 Cách chọn giá trị hạ quyền theo kiểu của trường vai trò 19
3.6 Điểm của thao tác `PUT /users/{username}/role` trên reports-api . 20

4.1 Các ứng dụng mục tiêu 23
4.2 Thiết lập chạy theo ứng dụng 26

5.1 Kết quả phát hiện và báo động giả theo lượt chạy 28
5.2 Phân loại kết quả theo lượt chạy 28
5.3 Hạng của thao tác chuyển trạng thái thật trên các ứng dụng mục tiêu 30

A.1 Các endpoint trong bộ ba do công cụ dựng trên bốn ứng dụng mục tiêu 39

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
API	Application Programming Interface	Giao diện lập trình ứng dụng
BFLA	Broken Function Level Authorization	Lỗi phân quyền cấp chức năng
BOLA	Broken Object Level Authorization	Lỗi phân quyền cấp đối tượng
CSRF	Cross-Site Request Forgery	Giả mạo yêu cầu liên trang
CVE	Common Vulnerabilities and Exposures	Mã định danh lỗ hổng
CVSS	Common Vulnerability Scoring System	Thang điểm mức nghiêm trọng của lỗ hổng
CWE	Common Weakness Enumeration	Danh mục điểm yếu phần mềm
FN	False Negative	Bỏ sót
FP	False Positive	Báo động giả
HTTP	Hypertext Transfer Protocol	Giao thức truyền siêu văn bản
IDOR	Insecure Direct Object Reference	Tham chiếu trực tiếp đối tượng không an toàn
INCITS	International Committee for Information Technology Standards	Ủy ban quốc tế về tiêu chuẩn công nghệ thông tin
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu
JWT	JSON Web Token	Chuẩn token dạng JSON dùng cho xác thực
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MIO	Many Independent Objective	Thuật toán nhiều mục tiêu độc lập
MR	Metamorphic Relation	Quan hệ biến hình
MR-A1	Metamorphic Relation A1	Quan hệ biến hình: phiên cũ phải bị từ chối sau khi hạ quyền
Oracle	Test Oracle	Cơ chế phán quyết kết quả ca kiểm thử là đúng hay có lỗi

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
OWASP	Open Worldwide Application Security Project	Dự án mở toàn cầu về an toàn ứng dụng
RBAC	Role-Based Access Control	Kiểm soát truy cập theo vai trò
REST	Representational State Transfer	Kiểu kiến trúc dịch vụ web
SMARTS	Smart Sampling	Thuật toán tìm kiếm mặc định của EvoMaster ở chế độ hộp đen
TN	True Negative	Âm tính thật
TP	True Positive	Phát hiện đúng

Chương 1

Giới thiệu

Khi quyền của người dùng thay đổi, máy chủ có thể vẫn chấp nhận thông tin xác thực đã cấp trước đó cho thao tác đã mất quyền. Chương này giới thiệu bài toán phát hiện lỗ hổng này, cùng mục tiêu, phạm vi và đóng góp của khóa luận.

1.1 Bối cảnh và động lực nghiên cứu

OWASP API Security Top 10 năm 2023 xếp lỗ hổng phân quyền ở mức đối tượng và mức chức năng vào nhóm rủi ro hàng đầu [1]. Khi hệ thống yêu cầu thu hồi quyền có hiệu lực ngay, nguy cơ an ninh phát sinh nếu phiên cũ vẫn giữ được quyền này. Hiện tượng máy chủ tiếp tục chấp nhận thao tác theo quyền đã thu hồi được gọi là lỗ hổng phân quyền có trạng thái (Stateful Broken Access Control).

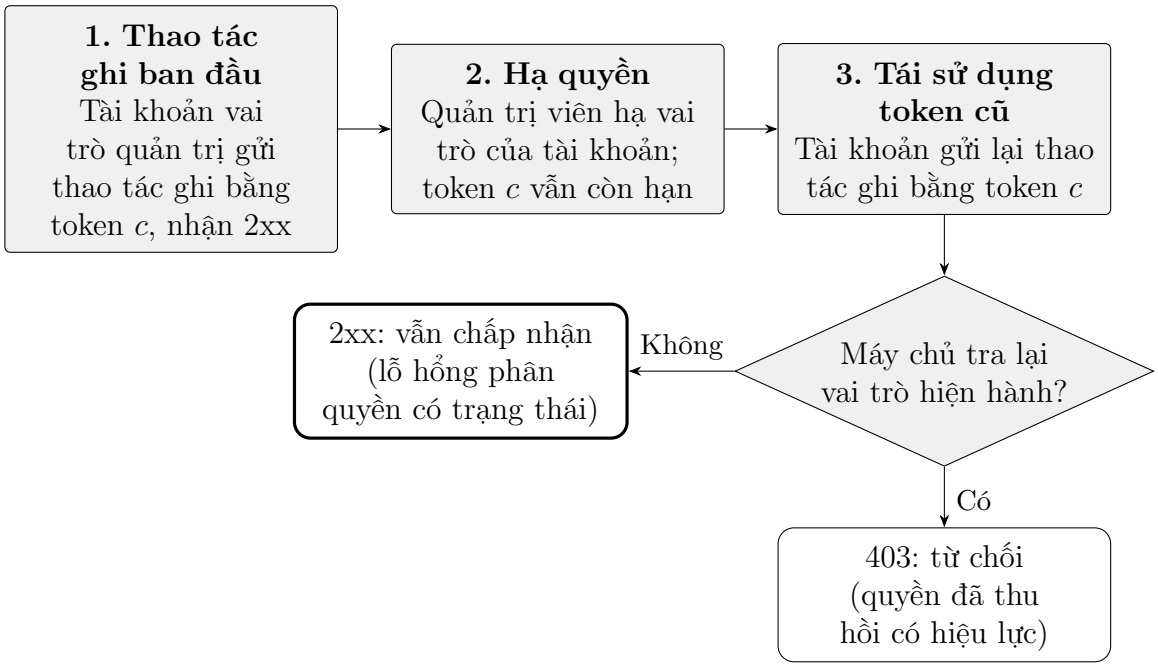
Lỗi này đã xuất hiện ở các dự án mã nguồn mở. Trình quản lý tải pyLoad mang lỗ hổng CVE-2026-41133 với điểm CVSS 8,8: ứng dụng chép vai trò vào phiên lúc đăng nhập và tiếp tục cấp quyền theo bản sao đó sau khi quản trị viên đổi vai trò [2]. Máy chủ lưu trữ gói phần mềm (artifact registry) artifact-keeper ghi cờ quản trị vào token JWT lúc đăng nhập, nên quản trị viên bị hạ quyền vẫn giữ quyền quản trị tới khi token hết hạn sau 30 phút; nhà phát triển đã vá lỗi này [3]. Hai ứng dụng dùng hai cơ chế xác thực khác nhau nhưng có chung một nguyên nhân: quyền được đọc từ bản sao cấp lúc đăng nhập thay vì từ trạng thái hiện tại.

EvoMaster áp dụng kiểm thử dựa trên tìm kiếm để tự động sinh ca kiểm thử cho REST API [4]. Với lỗi phân quyền, mã trạng thái của một yêu cầu HTTP thường không đủ để biết kết quả có đúng chính sách truy cập hay không. Các oracle phân quyền, tức cơ chế phán quyết kết quả kiểm thử, hiện có trong EvoMaster đối chiếu hành vi giữa các tài khoản có quyền khác nhau [5]. Các oracle này xét mỗi lời gọi tại một thời điểm, nên một lời gọi thành công bằng token cũ sau khi tài khoản bị hạ quyền vẫn trông như một lời gọi hợp lệ. Khi nhánh chính của EvoMaster được chạy trên một ứng dụng mang lỗi này, lưu lượng HTTP có bốn lời gọi như vậy nhận mã 200, nhưng không lời gọi nào bị báo lỗi (Mục 5.6).

1.2 Phát biểu bài toán

Trong phạm vi khóa luận, lỗ hổng phân quyền có trạng thái xảy ra khi hệ thống tiếp tục chấp nhận một thao tác phụ thuộc vào quyền đã thu hồi. Thao tác đó được gửi bằng thông tin xác thực cấp trước lần thay đổi quyền. Chẳng hạn, sau khi một tài khoản bị hạ xuống vai trò thấp hơn, máy chủ vẫn chấp nhận token hoặc cookie cấp trước đó cho thao tác không còn được phép. Tùy chính sách thu hồi quyền và cơ chế xác thực của hệ thống, hiện tượng này có thể thuộc điểm yếu CWE-1464 (Improper Session Invalidation), tức hệ thống không vô hiệu hóa phiên khi cần [6]. Hình 1.1 minh họa ba giai đoạn của lỗ hổng: thao tác ghi ban đầu, thao tác hạ quyền và tái sử dụng token cũ.

Khóa luận tập trung tự động phát hiện việc máy chủ tiếp tục chấp nhận quyền đã thu hồi. Hành vi này không thể phán quyết chỉ từ một yêu cầu HTTP riêng lẻ. Phán quyết cần kết quả của cùng thao tác trước và sau khi thay đổi quyền, cùng dấu hiệu cho thấy quyền đã thật sự bị thu hồi.



Hình 1.1: Vòng đời lỗ hổng phân quyền có trạng thái

1.3 Mục tiêu và phạm vi nghiên cứu

Khóa luận mở rộng EvoMaster để tự động phát hiện các vi phạm phân quyền có trạng thái thuộc quan hệ biến hình MR-A1. Phần mở rộng là một oracle mới, gọi là Oracle 970 theo mã lỗi gán cho vi phạm. Phần mở rộng chạy trong pha kiểm thử bảo mật, sau pha tìm kiếm, nên thuật toán tìm kiếm của công cụ được giữ nguyên.

Về phạm vi, khóa luận chỉ xét quan hệ MR-A1, trong đó quyền thực hiện một thao tác được kiểm tra trước và sau khi vai trò của chủ thể bị hạ. Kịch bản kiểm thử MR-A1 được dựng từ đặc tả OpenAPI, cấu hình xác thực, các quan sát HTTP và kho lưu trữ ca kiểm thử của EvoMaster. Phương pháp chỉ áp dụng khi có đủ ba điều kiện: thao tác hạ quyền thực hiện được qua REST API, hệ thống cho phép đăng nhập lại để lấy thông tin xác thực mới sau khi hạ quyền, và hệ thống từ chối thao tác không đủ quyền bằng mã 403. Các tình huống không thỏa các điều kiện này nằm ngoài phạm vi đánh giá. Chẳng hạn, thao tác xóa chủ thể không để lại tài khoản nào để đăng nhập lại.

1.4 Đóng góp của khóa luận

Khóa luận có bốn đóng góp chính:

1. **Quan hệ biến hình MR-A1.** Định nghĩa quan hệ MR-A1 để đối chiếu hành vi phân quyền trước và sau thao tác hạ quyền, với thông tin xác thực giữ cố định. Quan hệ được cài đặt bằng bộ ba thao tác gồm thao tác gốc (source), thao tác chuyển trạng thái (transition) và thao tác kiểm chứng (follow-up), cùng điều kiện tiền đề.

2. **Tự động nhận diện thao tác chuyển trạng thái.** Xếp hạng thao tác hạ quyền từ đặc tả OpenAPI bằng mô hình Naive Bayes; cùng mô hình này chọn trường vai trò và trường định danh tài khoản nạn nhân.

3. **Kiểm chứng bằng bước đăng nhập lại.** Tài khoản nạn nhân lấy token mới và chạy lại chuỗi lời gọi của mình; bộ ba bị loại nếu token mới không bị từ chối.

4. **Đánh giá thực nghiệm.** Đánh giá Oracle 970 trên bốn ứng dụng. Hai ứng dụng reports-api và vampi-seeded mang lỗi tự gieo. Hai ứng dụng mã nguồn mở mang lỗi thật: pyLoad với lỗ hổng CVE-2026-41133, và artifact-keeper với một lỗi đã được nhà phát triển vá. Trên các biến thể có lỗ hổng, Oracle 970 phát hiện vi phạm ở 34/50 lượt, gồm 5/5 lượt trên pyLoad và 4/5 lượt trên artifact-keeper. Trong 48 lượt chạy trên biến thể đã vá, 30 lượt dựng được bộ ba và Oracle 970 không báo động giả ở lượt nào.

1.5 Bố cục khóa luận

Sau Chương 2 về cơ sở lý thuyết và các công trình liên quan, Chương 3 định nghĩa quan hệ MR-A1, cách dựng kịch bản kiểm thử và cách nhận diện thao tác hạ quyền. Cách tích hợp Oracle 970 vào EvoMaster, các ứng dụng mục tiêu và thiết lập thực nghiệm nằm ở Chương 4. Chương 5 đánh giá khả năng phát hiện vi phạm, khả năng xếp hạng thao tác chuyển trạng thái và các giới hạn quan sát được, còn Chương 6 tổng kết kết quả và đề xuất hướng phát triển.

Chương 2

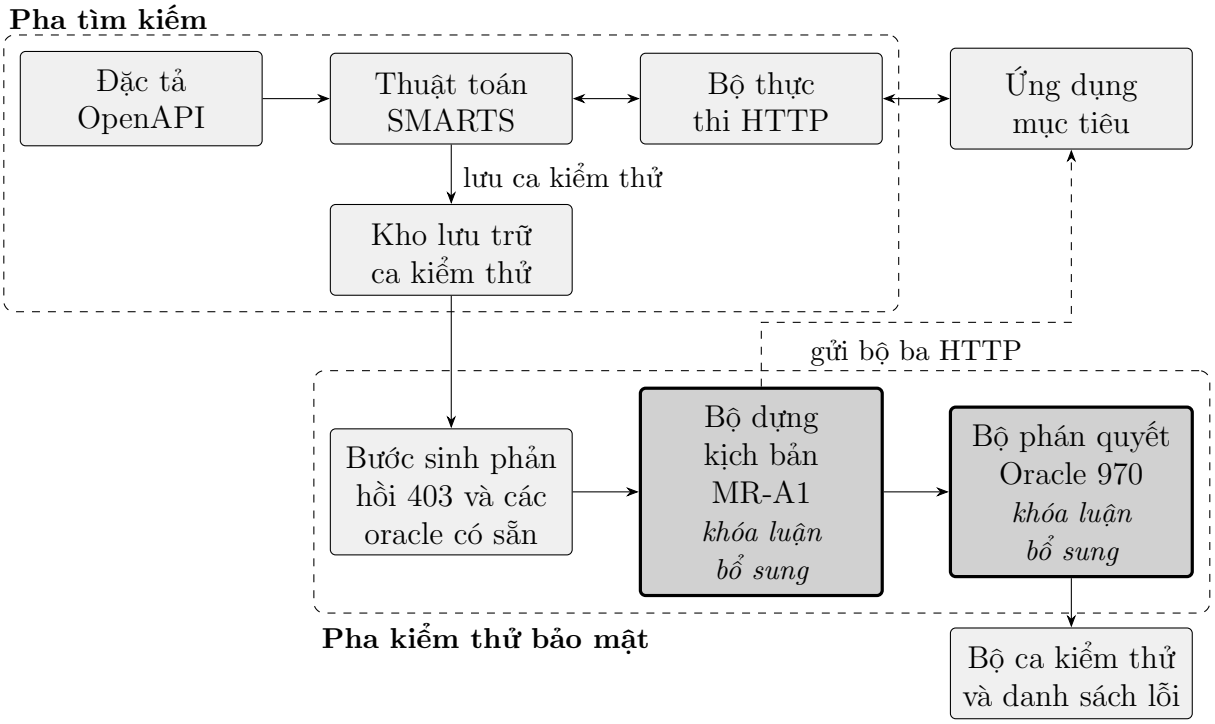
Cơ sở lý thuyết và nghiên cứu liên quan

Phát hiện vi phạm phân quyền sau khi quyền thay đổi đòi hỏi kết hợp ba nền tảng: kiểm thử REST API dựa trên tìm kiếm, kiểm thử biến hình và kiểm soát truy cập. Chương này trình bày ba nền tảng đó và đối chiếu các công trình liên quan để xác định khoảng trống mà khóa luận nhắm tới.

2.1 Kiểm thử REST API dựa trên tìm kiếm

Kiểm thử dựa trên tìm kiếm (search-based software testing) coi việc sinh ca kiểm thử là một bài toán tối ưu hóa. EvoMaster áp dụng cách tiếp cận này cho REST API: công cụ sinh các chuỗi yêu cầu HTTP, rồi dựa vào phản hồi thu được khi thực thi để chọn ca kiểm thử giữ lại [4]. Khóa luận dùng chế độ hộp đen, trong đó EvoMaster chỉ đọc đặc tả OpenAPI và gửi yêu cầu HTTP tới ứng dụng đang chạy [7]. Thuật toán tìm kiếm mặc định của EvoMaster 6.1.1 tùy theo chế độ chạy. Chế độ hộp trắng dùng Many Independent Objective (MIO), thuật toán chọn và biến đổi ca kiểm thử theo từng mục tiêu [8]. Chế độ hộp đen dùng SMARTS (Smart Sampling): mỗi vòng lặp, thuật toán lấy mẫu một ca kiểm thử mới và không biến đổi các ca đã có [9]. Với xác suất 0,95, ca mới được dựng theo chiến lược có cấu trúc, chẳng hạn gọi `POST` tạo một tài nguyên rồi gọi `DELETE` lên chính tài nguyên đó; các ca còn lại là chuỗi lời gọi ngẫu nhiên. Các lượt chạy trong khóa luận giữ thuật toán mặc định, nên đều dùng SMARTS. Kho lưu trữ ca kiểm thử giữ những ca đại diện cho các mục tiêu đã đạt; bộ ca kiểm thử cuối và các pha sau đều lấy đầu vào từ kho này.

Pha kiểm thử bảo mật của EvoMaster chạy sau pha tìm kiếm [5]. Pha này dùng các ca kiểm thử và kết quả thực thi trong kho lưu trữ để dựng thêm các ca kiểm thử nhắm vào lỗi bảo mật, không cần thay đổi thuật toán tìm kiếm. Hình 2.1 biểu diễn chu trình kiểm thử và vị trí pha kiểm thử bảo mật.



Hình 2.1: Chu trình kiểm thử EvoMaster với pha kiểm thử bảo mật

2.2 Kiểm thử biến hình

Oracle kiểm thử (test oracle) là cơ chế đối chiếu kết quả thực thi với hành vi kỳ vọng hoặc một tiêu chí kiểm tra xác định trước [10]. Khi không thể xây dựng một oracle đầy đủ hoặc chi phí xây dựng quá lớn, quá trình kiểm thử gặp bài toán thiếu oracle (test oracle problem). Kiểm thử biến hình (metamorphic testing) hỗ trợ kiểm thử khi thiếu oracle bằng cách đối chiếu quan hệ kỳ vọng giữa những lần thực thi có liên quan [11]. Một quan hệ biến hình (metamorphic relation) phát biểu mối liên hệ kỳ vọng giữa đầu vào và đầu ra của các lần thực thi đó. Khi các quan sát thỏa điều kiện áp dụng nhưng vi phạm quan hệ, kết quả là bằng chứng về hành vi không phù hợp với thuộc tính được kiểm tra [12].

2.3 Lỗ hổng kiểm soát truy cập trong REST API

Kiểm soát truy cập (access control) là cơ chế xác định một chủ thể được phép thực hiện những thao tác nào trên từng tài nguyên của hệ thống. Lỗ hổng phân quyền ở mức đối tượng (Broken Object Level Authorization, BOLA) phát sinh khi API

không kiểm tra đủ quyền của chủ thể đối với đối tượng được yêu cầu. Lỗ hổng phân quyền ở mức chức năng (Broken Function Level Authorization, BFLA) phát sinh khi API không kiểm tra đủ quyền gọi một chức năng. Cả hai lớp lỗi nằm trong OWASP API Security Top 10 [1]. Lớp lỗi khóa luận xét gần nhất với mục API5 (BFLA) của danh sách này: sau khi bị hạ quyền, tài khoản đã là người dùng thường nhưng vẫn gọi được chức năng của quản trị viên. Điểm khác là lỗi chỉ lộ ra với thông tin xác thực cấp trước lần hạ quyền.

Xác thực (authentication) xác định người gọi là ai, còn phân quyền (authorization) quyết định người gọi được làm gì. Sau khi đăng nhập, máy chủ cấp cho client một thông tin xác thực để gửi kèm các yêu cầu sau. Các ứng dụng trong khóa luận dùng một trong hai cách. Với token JWT, máy chủ ghi tên tài khoản và các thuộc tính như vai trò vào phần claim của token rồi ký, nên ở các yêu cầu sau máy chủ có thể kiểm chữ ký mà không tra cơ sở dữ liệu [13]. Với phiên phía máy chủ, client giữ một cookie chứa mã phiên, còn dữ liệu của phiên nằm trên máy chủ. Ở cả hai cách, nếu máy chủ quyết định quyền theo vai trò đã chép vào token hay phiên lúc đăng nhập, lần đổi vai trò sau đó chưa có hiệu lực với thông tin xác thực đã cấp cho tới khi nó hết hạn. Có hai hướng khắc phục: tra lại vai trò hiện tại ở mỗi yêu cầu, hoặc thu hồi các thông tin xác thực cấp trước lần đổi vai trò.

Các oracle BOLA và BFLA hiện có đối chiếu phản hồi của các tài khoản có quyền khác nhau. Khác với các oracle này, khóa luận xét trường hợp cùng một chủ thể vẫn được chấp nhận sau khi quyền cần thiết đã bị thu hồi. Hiện tượng này được gọi là lỗ hổng phân quyền có trạng thái (Stateful Broken Access Control). Với hệ thống có chính sách thu hồi quyền có hiệu lực ngay, các yêu cầu tiếp theo phụ thuộc vào quyền vừa bị thu hồi phải bị từ chối. Chuẩn kiểm soát truy cập theo vai trò RBAC mô tả các lệnh thu hồi như hủy gán người dùng khỏi vai trò và thu hồi quyền khỏi vai trò [14].

Thay đổi vai trò có áp dụng ngay cho phiên đang hoạt động hay không tùy thuộc vào chính sách và cơ chế thực thi của từng hệ thống. Nếu chính sách yêu cầu chấm dứt hoặc cập nhật phiên mà máy chủ vẫn chấp nhận thông tin xác thực cũ sau khi thu hồi quyền, hành vi này có thể thuộc điểm yếu CWE-1464 (Mục 1.2).

2.4 Các công trình nghiên cứu liên quan

RESTler là một công cụ kiểm thử REST API khác, chủ yếu nhắm tới độ phủ hành vi và lỗi chức năng. Công cụ này sinh các chuỗi lời gọi có trạng thái, kèm một số bộ kiểm tra lỗi [15]. Với lỗi phân quyền, Şahin, Zhang và Arcuri bổ sung vào EvoMaster một nhóm oracle an toàn dưới dạng pha hậu xử lý, so sánh nhiều tài khoản tại cùng một thời điểm để phát hiện lỗi BOLA và BFLA [5, 16]. Nhóm oracle này có sẵn trong EvoMaster 6.1.1 và là phương án đối sánh (baseline) của khóa luận.

Ngoài EvoMaster, BACScan phát hiện lỗ hổng kiểm soát truy cập trong ứng dụng web ở chế độ hộp đen; oracle của công cụ dựa vào phản hồi của thao tác để xác định một thao tác đọc hoặc sửa trái phép đã xảy ra hay chưa [17]. GraphQLer kiểm thử API GraphQL bằng các chuỗi lời gọi nhiều bước: phát lại yêu cầu của một người dùng bằng thông tin xác thực của người dùng khác để tìm lỗi tham chiếu trực tiếp đối tượng không an toàn (IDOR), và chuỗi tạo, xóa, đọc để tìm lỗi dùng tài nguyên sau khi xóa [18]. Chuỗi tạo, xóa, đọc có yếu tố thời gian ở phía tài nguyên. Khóa luận xét yếu tố thời gian ở phía quyền: quyền bị thu hồi trong khi thông tin xác thực cấp trước đó vẫn được dùng.

Trong EvoMaster, chưa có oracle nào kết hợp đồng thời ba cơ chế của MR-A1: tự động dựng thao tác thu hồi quyền, dùng lại token cấp trước khi thu hồi, và phán quyết dựa trên quan sát trước và sau khi thay đổi quyền. Nhận định này dựa trên mã nguồn upstream của EvoMaster (bản do nhóm phát triển EvoMaster duy trì) tại commit `ab983ba338`; Mục 5.6 kiểm thêm bằng cách chạy bản upstream này trên reports-api. Chương 3 định nghĩa quan hệ MR-A1 và cách cài đặt quan hệ này trong EvoMaster.

Chương 3

Phương pháp đề xuất

Phương pháp đề xuất phát hiện lỗ hổng phân quyền có trạng thái bằng cách đối chiếu kết quả của cùng một thao tác trước và sau khi quyền của chủ thể bị thu hồi. Chương này trình bày phương pháp từ quan hệ biến hình MR-A1 đến kịch bản kiểm thử chạy được trong EvoMaster.

3.1 Tổng quan Oracle 970

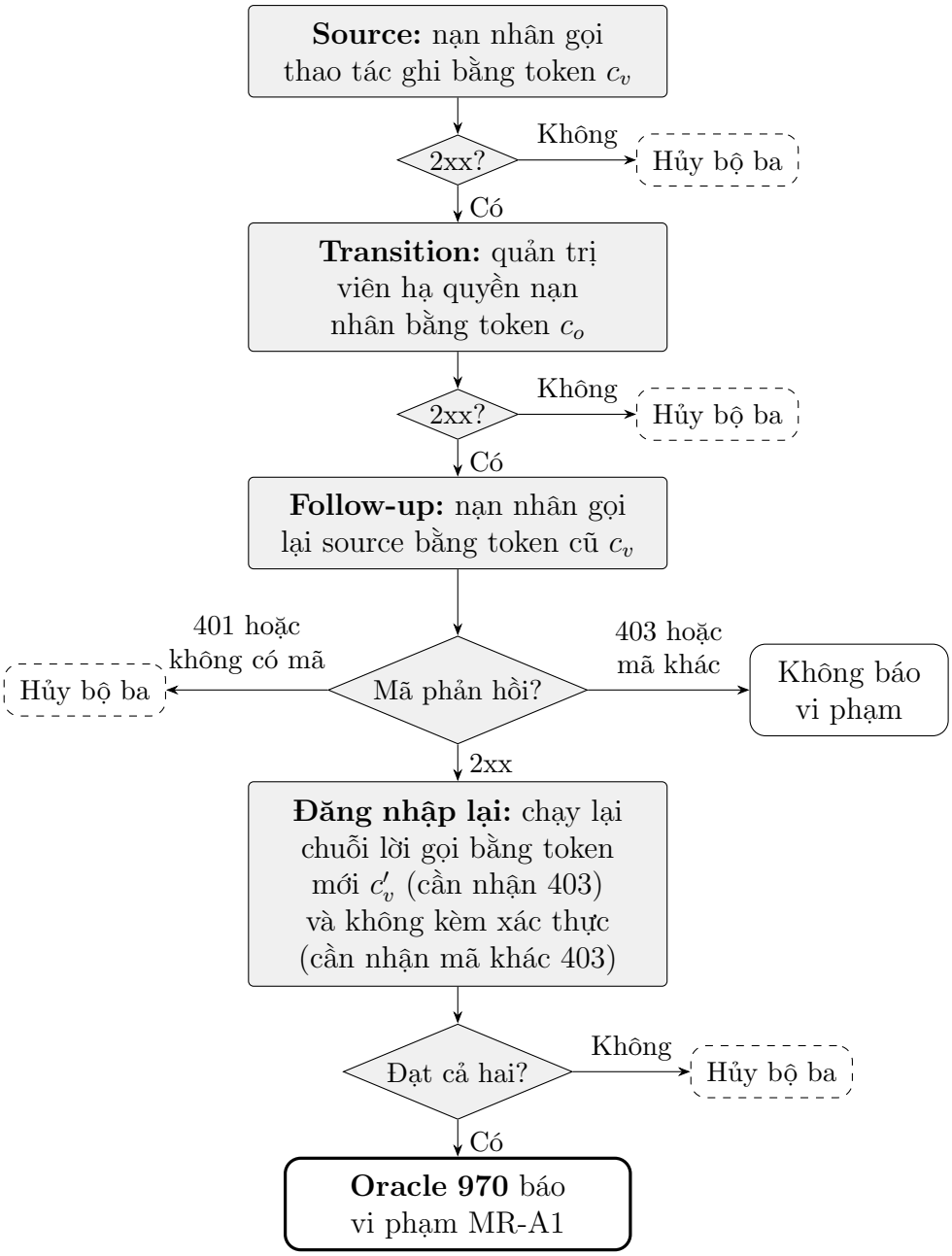
Trong kiểm thử phân quyền REST API, bài toán thiếu oracle (Mục 2.2) xuất hiện vì mã trạng thái HTTP không cho biết thao tác có được phép theo chính sách hay không. Máy chủ có thể trả `200 OK` cho một thao tác mà chính sách phân quyền cấm.

Oracle 970 chạy trong pha kiểm thử bảo mật của EvoMaster 6.1.1 và dùng cùng kiến trúc với các oracle an toàn có sẵn của công cụ (Mục 2.4). Pha này lấy đầu vào từ kho lưu trữ ca kiểm thử (Mục 2.1). Số 970 là mã lỗi phần mở rộng gán cho vi phạm MR-A1: EvoMaster 6.1.1 dành dải mã 9xx cho các oracle thử nghiệm, và 970 còn trống trong dải này [9].

Phương pháp tách làm hai thành phần: bộ dựng kịch bản kiểm thử tạo các bộ ba thao tác (source, transition, follow-up), còn bộ phán quyết đánh giá các bộ ba đã thực hiện thành công. Cách tách này phân biệt được lượt không dựng được bộ ba hợp lệ với lượt có bộ ba nhưng không có vi phạm. Hình 3.1 trình bày trình tự thực hiện bộ ba và điều kiện kích hoạt Oracle 970.

3.2 Quan hệ biến hình MR-A1

MR-A1 là một quan hệ biến hình (Mục 2.2) giữa hai lần thực hiện một thao tác, trước và sau khi thu hồi quyền. Trong tên MR-A1, chữ A chỉ nhóm quan hệ phân quyền (authorization) và số 1 chỉ quan hệ đầu tiên của nhóm. Các quan hệ trong nhóm có cùng dạng và khác nhau ở cách chủ thể mất quyền: A1 xét trường hợp



Hình 3.1: Trình tự thực hiện bộ ba thao tác và kích hoạt Oracle 970

vai trò bị hạ, còn chuyển quyền sở hữu tài nguyên và quyền hết hạn theo thời gian thuộc các quan hệ khác (Mục 6.3).

MR-A1 xét một chủ thể p, tức tài khoản được kiểm tra, có quyền thực hiện thao tác trước khi bị hạ quyền. Chủ thể này gửi yêu cầu bằng thông tin xác thực c định danh p, có thể là token hoặc cookie phiên. Thao tác a là một lời gọi REST API dùng phương thức ghi và có kiểm soát phân quyền, chẳng hạn thao tác cập nhật báo cáo mà chỉ quản trị viên được làm. Trạng thái s là cấu hình phân quyền của

hệ thống tại lúc quan sát. Một tài khoản quản trị đổi trạng thái này bằng thao tác T (transition) để hạ vai trò hoặc thu hồi quyền của p.

Hàm quan sát $\mathrm{Obs}(a, c, s)$ lấy kết quả từ mã trạng thái HTTP khi chủ thể dùng c gọi thao tác a tại trạng thái s [19]. Bảng 3.1 nêu ba giá trị quan sát và mã trạng thái tương ứng. Giá trị ALLOWED không xác nhận trạng thái dữ liệu nghiệp vụ sau thao tác. Oracle phán quyết dựa trên mã 403. Mã 404 không được tính là lời từ chối, vì tài nguyên không tồn tại cũng trả về mã này. Trong 98 lượt chạy ở Chương 5, không bước follow-up nào nhận mã 404.

Mã `401` ở bước follow-up nằm ngoài phạm vi phán quyết, vì mã này không phân biệt được hai trường hợp: máy chủ từ chối vì quyền, hay máy chủ không chấp nhận thông tin xác thực. Trên các ứng dụng mục tiêu, nhóm đường dẫn `/api/*` của pyLoad trả 401 cho phiên không có quyền quản trị, và VAmPI trả 401 khi token không hợp lệ. Bản vá của artifact-keeper thu hồi quyền bằng cách vô hiệu hóa phiên và trả 401 cho token đã bị thu hồi (Mục 5.4). Nếu coi 401 là DENIED, một token hết hạn giữa bộ ba sẽ được tính là một lần thu hồi quyền đúng.

Bảng 3.1: Ánh xạ giá trị quan sát sang mã trạng thái HTTP

Giá trị	Mã HTTP	Ý nghĩa
ALLOWED	2xx	Máy chủ chấp nhận lời gọi ở tầng HTTP.
DENIED	403	Máy chủ từ chối lời gọi trên cơ sở phân quyền.
INCONCLUSIVE	401	Phiên hết hạn hoặc chưa xác định được quyền.

Gọi $s' = T(s)$ là trạng thái sau khi thực hiện T. Khi c được giữ cố định qua hai lần quan sát, MR-A1 có dạng Công thức (3.1).

$$\mathrm{Obs}(a, c, s) = \textsc{allowed} \ \wedge \ \mathit{Pre}(a, T) \implies \mathrm{Obs}(a, c, s') = \textsc{denied}. \tag{3.1}$$

Trong đó, $\mathit{Pre}(a, T)$ là điều kiện tiền đề: thao tác a cần đúng quyền mà T thu hồi của chủ thể đang xét. Khi vế trái đúng mà kết quả ở trạng thái s' vẫn là ALLOWED, MR-A1 bị vi phạm và oracle báo lỗi.

Trong thuật ngữ kiểm thử biến hình, lần thực thi nguồn là yêu cầu a gửi bằng c ở trạng thái s. Phép biến đổi giữ nguyên yêu cầu và thông tin xác thực, chỉ đổi trạng thái của hệ thống bằng T. Lần thực thi tiếp theo gửi lại đúng yêu cầu đó bằng cùng c ở trạng thái s' và không đăng nhập lại. Lỗi cần tái hiện nằm ở phiên

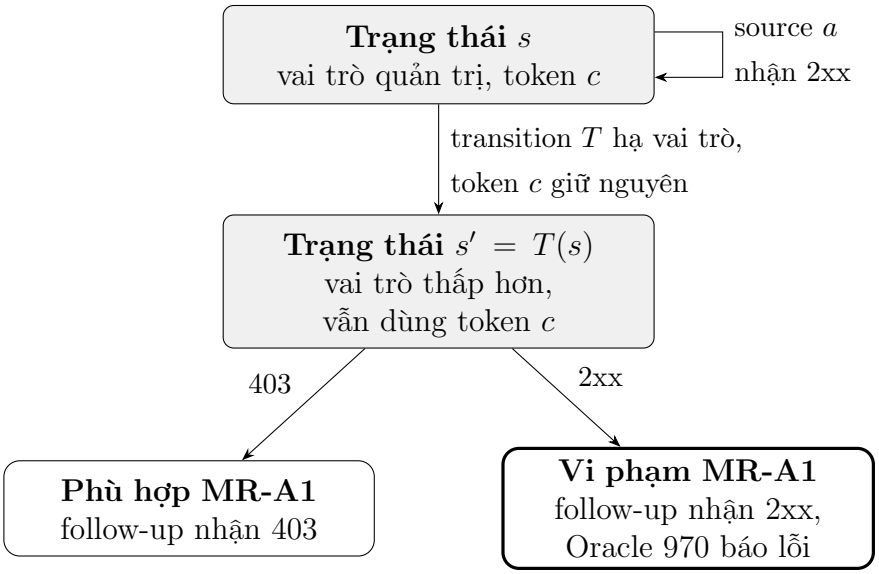
cũ: phiên mở trước lúc hạ quyền vẫn giữ quyền đã bị thu hồi.

Điều kiện tiền đề loại các thao tác không phụ thuộc vào quyền bị thu hồi để giảm báo động giả. Chẳng hạn, thao tác cập nhật hồ sơ cá nhân dùng chung cho nhiều vai trò; sau khi tài khoản bị hạ quyền, máy chủ vẫn chấp nhận thao tác này là đúng. Do chuẩn RBAC để ngỏ cách cập nhật quyền trên phiên đang hoạt động [14], yêu cầu từ chối ngay sau khi hạ quyền là chính sách riêng của từng hệ thống mục tiêu.

Bảng 3.2 đối chiếu MR-A1 với nhóm oracle so sánh chéo tài khoản (differential oracle) của Şahin và cộng sự [5], dùng để phát hiện lỗi BOLA và BFLA. Hình 3.2 biểu diễn hai trạng thái phân quyền cùng nhánh an toàn và nhánh vi phạm.

Bảng 3.2: Đối chiếu oracle so sánh chéo tài khoản và quan hệ MR-A1

Tiêu chí	Oracle so sánh chéo tài khoản	MR-A1 (theo thời gian)
So sánh theo	nhiều tài khoản tại một thời điểm	một tài khoản trước và sau khi hạ quyền
Thông tin xác thực	khác nhau theo tài khoản	giữ cố định và dùng lại
Nội dung kiểm tra	tài khoản không có quyền vẫn truy cập được	quyền đã thu hồi vẫn được chấp nhận



Hình 3.2: Hai trạng thái phân quyền trong MR-A1

3.3 Bộ ba thao tác và điều kiện tiền đề

Bộ dựng kịch bản kiểm thử chuyển quan hệ MR-A1 thành ca kiểm thử chạy được trên REST API, gọi là kịch bản kiểm thử MR-A1. Đầu vào của bộ dựng là các ca kiểm thử trong kho lưu trữ, gồm cả những ca nhận mã 403 do bước sinh phản hồi 403 có sẵn của EvoMaster tạo ra [9].

3.3.1 Cấu trúc bộ ba thao tác

Kịch bản kiểm thử MR-A1 gồm các lời gọi khởi tạo và bộ ba thao tác chính, chạy theo thứ tự sau:

1. Khởi tạo (setup): các lời gọi trong ca kiểm thử ban đầu của nạn nhân, chạy trước thao tác gốc để tạo tài nguyên cần thao tác. Bước này không bắt buộc nếu tài nguyên đã có sẵn trong hệ thống.
2. Source (thao tác gốc): tài khoản nạn nhân, trước khi bị hạ quyền, gọi một thao tác ghi có kiểm soát phân quyền và nhận mã `2xx`.
3. Transition (thao tác chuyển trạng thái): tài khoản quản trị hạ vai trò hoặc thu hồi quyền tương ứng của nạn nhân.
4. Follow-up (thao tác kiểm chứng): nạn nhân gọi lại thao tác source với cùng tham số và cùng token cũ.

Bảng 3.3: Kịch bản kiểm thử MR-A1 trên reports-api

#	Tài khoản	Lời gọi	Bản lỗi	Bản vá
1	`alice`	Khởi tạo: `POST /reports`	201	201
2	`alice`	Source: `PUT /reports/{id}` trên báo cáo vừa tạo	200	200
3	`superadmin`	Transition: `PUT /users/alice/role`, đặt `role` thành `VIEWER`	200	200
4	`alice`	Follow-up: lặp lại lời gọi 2 với cùng token và cùng tham số	200	403

Follow-up dùng lại được token cũ vì EvoMaster đăng nhập mỗi tài khoản một lần ở đầu ca kiểm thử, rồi dùng token hoặc cookie phiên đó cho mọi thao tác sau.

Bảng 3.3 là một kịch bản kiểm thử MR-A1 do công cụ dựng trên reports-api (Mục 4.2.1). Tài khoản `alice` có vai trò quản trị, còn `superadmin` là tài khoản quản trị hạ quyền `alice` xuống `VIEWER`.

3.3.2 Điều kiện tiền đề và cách chọn source

Bộ dựng kiểm tra điều kiện tiền đề ngay khi chọn source. Source phải là thao tác ghi, xác định theo phương thức HTTP (`PUT`, `PATCH`, `POST`), và khác đường dẫn với thao tác chuyển trạng thái. Bộ dựng chọn một thao tác ghi làm source khi kho lưu trữ có đồng thời hai quan sát sau:

1. Thao tác này từng nhận mã 403 ít nhất một lần. Ngoài các lần 403 xuất hiện trong pha tìm kiếm, bước sinh phản hồi 403 tạo thêm quan sát này bằng cách gửi lại một ca kiểm thử thành công bằng tài khoản khác.
2. Tài khoản nạn nhân từng thực hiện thành công thao tác này với mã `2xx`.

Cặp phản hồi 403 và `2xx` này chỉ là dấu hiệu ban đầu, vì lời từ chối có thể đến từ quyền sở hữu tài nguyên hoặc một điều kiện khác. Phân loại theo phương thức cho phép source là một lời gọi `POST` chỉ đọc dữ liệu. Trên pyLoad, source `POST /json/status` trả trạng thái máy chủ và bị chặn bằng quyền `LIST`, quyền mà tài khoản mất khi bị hạ.

Bộ dựng ưu tiên `PUT` và `PATCH`: follow-up lặp lại source trên cùng tài nguyên, và hai phương thức này thường gọi lại được nhiều lần. Bộ dựng chỉ xét `POST` khi không thao tác `PUT` hay `PATCH` nào có lời gọi thành công bằng tài khoản đã đăng nhập. `DELETE` không được chọn làm source, vì sau khi xóa, follow-up không còn tài nguyên để gọi lại.

3.3.3 Bộ ba bị hủy và bước đăng nhập lại

Bộ ba bị hủy và loại khỏi tập đánh giá khi source hoặc transition trả mã ngoài nhóm `2xx`, hoặc khi follow-up không ghi được mã trạng thái hay nhận mã `401`. Sau mỗi lần hủy, bộ dựng chọn tổ hợp mới theo các luật ở Mục 3.3.4. Với các bộ

ba còn lại, mã của follow-up quyết định bước tiếp theo: 403 nghĩa là bộ ba phù hợp MR-A1, còn `2xx` dẫn tới bước đăng nhập lại. Với mã khác, như 404 hay 500, bộ ba được giữ và không bị báo vi phạm; các lượt chạy của khóa luận không gặp trường hợp này.

Trong một số lần thử, mọi tài khoản quản trị đều bị từ chối khi hạ quyền, dù source của nạn nhân vẫn thành công. Tình huống này xảy ra khi pha tìm kiếm đã hạ quyền các tài khoản quản trị nhưng chưa hạ quyền tài khoản nạn nhân. Bộ dựng khi đó dùng quyền quản trị còn hiệu lực của tài khoản nạn nhân để khôi phục vai trò cho tài khoản quản trị rồi thử lại. Lần khôi phục là một ca kiểm thử riêng, chạy trước bộ ba mới: nạn nhân gọi chính thao tác chuyển trạng thái để nâng quyền cho tài khoản quản trị. Ca kiểm thử này không tự tạo ra cảnh báo, vì bộ ba dựng sau đó vẫn phải qua mọi điều kiện hủy và bước đăng nhập lại. Trên pyLoad, ở mọi lượt chạy, các tài khoản quản trị đều bị từ chối khi hạ quyền nạn nhân `pyload`; `pyload` nâng quyền `alice` trở lại, rồi `alice` hạ quyền `pyload` để dựng bộ ba. Bước khôi phục chỉ đảo được việc hạ quyền: tài khoản đã bị xóa trong pha tìm kiếm không phục hồi được.

Khi follow-up nhận `2xx`, bộ dựng chạy thêm bước đăng nhập lại trước khi chuyển bộ ba sang bộ phán quyết. Bước này kiểm tra transition có thu hồi quyền hay không. Tên endpoint chưa đủ để biết: một endpoint có tên gợi ý hạ quyền vẫn có thể nâng quyền, tùy giá trị vai trò gửi kèm.

Ở bước đăng nhập lại, bộ dựng dùng token mới của tài khoản nạn nhân để chạy lại chuỗi lời gọi của nạn nhân trong ca kiểm thử gốc. Chuỗi này gồm các lời gọi khởi tạo dữ liệu và thao tác source. Do EvoMaster dừng chuỗi tại lời gọi tạo tài nguyên đầu tiên thất bại, bộ dựng đọc mã trạng thái của lời gọi cuối cùng được thực thi. Mã 403 tại lời gọi này cho thấy token mới đã mất quyền ghi.

Khi nhận mã 403, bộ dựng chạy cùng chuỗi một lần nữa, không kèm thông tin xác thực. Nếu lần gọi này cũng nhận 403, mã 403 nhận được bằng token mới chưa chắc do quyền bị thu hồi. Bộ ba chỉ được giữ khi lần gọi không kèm xác thực trả về một mã khác 403, chẳng hạn 401. Mọi trường hợp còn lại, kể cả khi token mới vẫn nhận `2xx`, đều làm bộ ba bị hủy.

Với kịch bản ở Bảng 3.3 trên biến thể có lỗ hổng, bước đăng nhập lại chạy lại lời gọi 1 và 2 bằng token mới của `alice`. Lời gọi tạo báo cáo nhận 403 nên chuỗi dừng tại đó; lượt gửi không kèm thông tin xác thực nhận 401. Bộ ba được giữ và Oracle

970 báo vi phạm.

Bước đăng nhập lại chạy sau follow-up. Ở hệ thống giữ một phiên cho mỗi tài khoản, lần đăng nhập mới sẽ vô hiệu hóa token cũ mà follow-up cần. Token mới chỉ có được qua lần đăng nhập ở đầu ca kiểm thử, nên bước này chạy trong một ca kiểm thử riêng.

Công cụ hộp đen không quan sát trực tiếp được điều kiện tiền đề $Pre(a, T)$, vì quy tắc phân quyền nằm trong logic nội bộ của ứng dụng. Bộ dựng ước lượng điều kiện này bằng dấu hiệu HTTP ở hai thời điểm: khi chọn source và ở bước đăng nhập lại. Ở thời điểm thứ hai, mã 403 có thể đến từ một lời gọi khởi tạo thay vì từ chính source. Một cảnh báo của Oracle 970 là vi phạm MR-A1 khi hai giả định cùng đúng: mã 403 phản ánh quyết định phân quyền, và phiên mới mang quyền hiện hành. Nếu giả định thứ hai sai, tức token mới vẫn mang quyền cũ, bước đăng nhập lại có thể loại nhầm một bộ ba chứa lỗ hổng.

3.3.4 Thuật toán dựng kịch bản kiểm thử MR-A1

Thuật toán 3.1 duyệt các thao tác chuyển trạng thái theo thứ tự xếp hạng và ghép mỗi thao tác với các thao tác ghi ứng viên trong kho lưu trữ. Mỗi tổ hợp chạy trong giới hạn số lần thử, kèm bước khôi phục quyền khi thao tác chuyển trạng thái bị từ chối.

Trong thuật toán, c_v và c_o là thông tin xác thực của tài khoản nạn nhân v và tài khoản quản trị. Dòng 4 ước lượng $Pre(a, T)$ bằng hai quan sát khi chọn source. Ở dòng 8, bộ dựng áp các điều kiện hủy cho source, transition và follow-up, gồm cả bước đăng nhập lại khi follow-up nhận `2xx`. Bước khôi phục quyền nằm ở dòng 11; lần thử kế tiếp cho biết bước này có thành công hay không.

Ba trần giới hạn chi phí chạy: tổng $N = 20$ lần thử trong cả pha, $K = 5$ lần thử và $R = 3$ lần khôi phục quyền cho mỗi thao tác chuyển trạng thái. Thuật toán 3.1 là bản rút gọn: trong cài đặt, trần K không áp dụng cho thao tác chuyển trạng thái đã dựng được bộ ba và cho ứng viên cuối của danh sách. Với mỗi thao tác ghi ứng viên, bộ ba hợp lệ đầu tiên được đưa vào tập kết quả cho bộ phán quyết, kể cả khi follow-up nhận 403. Bộ dựng sau đó chuyển sang ứng viên kế tiếp; một lượt chạy có thể cho nhiều bộ ba.

Khi một lần thử thất bại, bộ dựng chọn bước kế tiếp dựa trên mã trạng thái của

Thuật toán 3.1 Dựng kịch bản kiểm thử MR-A1

Đầu vào: kho lưu trữ $\mathcal{A}$; giới hạn lần thử K, N; giới hạn khôi phục R

Đầu ra: tập $\mathcal{I}$ các bộ ba đã thực hiện

$n \leftarrow 0$; $\mathcal{I} \leftarrow \emptyset$
với mỗi thao tác chuyển trạng thái T theo thứ tự xếp hạng **thực hiện**
    $k \leftarrow 0$; $r \leftarrow 0$
    với mỗi thao tác ghi $a \neq T$ thỏa hai quan sát ở Mục 3.3.2 **thực hiện**
        trong khi $k < K$ **và** $n < N$ **thực hiện**
            $I \leftarrow$ [khởi tạo của $a,\ a(c_v),\ T(c_o, v),\ a(c_v)$]
            chạy I
            nếu I không bị hủy **thì**
                $\mathcal{I} \leftarrow \mathcal{I} \cup \{I\}$; **dừng**
            ngược lại nếu T nhận mã 401 hoặc 403 **và** $r < R$ **thì**
                dùng c_v khôi phục quyền cho tài khoản quản trị; $r \leftarrow r + 1$
            $k \leftarrow k + 1$; $n \leftarrow n + 1$
trả về $\mathcal{I}$

thao tác chuyển trạng thái (Bảng 3.4). Tài khoản quản trị nào từng gọi thành công thao tác chuyển trạng thái trong kho lưu trữ được thử trước. Các luật bỏ qua giúp bộ dựng không tiêu lượt thử vào những lần đã biết sẽ thất bại.

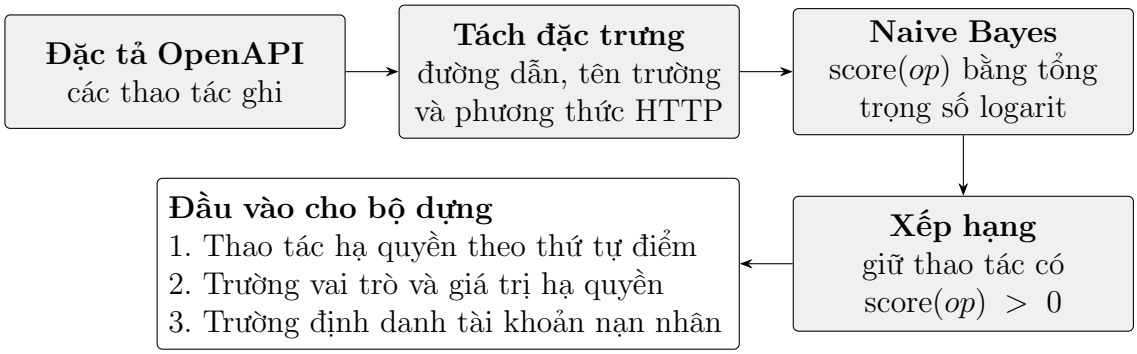
3.4 Tự động nhận diện và xếp hạng thao tác chuyển trạng thái

Số lần thử của Thuật toán 3.1 có giới hạn, nên bộ dựng thử các thao tác chuyển trạng thái theo thứ tự xếp hạng. Mô hình Naive Bayes lập thứ tự này bằng cách chấm điểm từng thao tác ghi trong đặc tả OpenAPI. Hình 3.3 tóm tắt quy trình nhận diện.

Naive Bayes là bộ phân loại xác suất dựa trên định lý Bayes, giả định các đặc trưng độc lập có điều kiện khi biết nhãn. Mô hình học từ 10.381 thao tác ghi của 230 REST API. Một thao tác mang nhãn chuyển trạng thái khi nhắm vào một tài khoản đã tồn tại hoặc chính người gọi, đổi vai trò, quyền hay cờ quản trị của tài khoản đó, và không thuộc loại chỉ cấp thêm quyền. Các thao tác còn lại, kể cả thao

Bảng 3.4: Luật thử lại và bỏ qua của bộ dựng

Tình huống	Bước tiếp theo
Transition nhận mã `4xx` hoặc `5xx` khác 401, 403, 404	Thử giá trị hạ quyền kế tiếp với cùng tài khoản quản trị
Transition nhận 401 hoặc 403	Thử tài khoản quản trị kế tiếp, rồi khôi phục quyền khi đã thử hết
Transition nhận `2xx`	Không dùng lại nạn nhân này cho thao tác chuyển trạng thái đang thử
Transition từ chối mọi lần thử trên một nạn nhân	Không thử lại cặp thao tác chuyển trạng thái và nạn nhân này trong pha
Transition nhận 404 sau source thành công, khi chưa dựng được bộ ba nào	Bỏ thao tác chuyển trạng thái này, chuyển sang ứng viên kế
Đường dẫn của ứng viên và đường dẫn cha chỉ nhận 404 trong kho lưu trữ	Loại ứng viên trước khi thử



Hình 3.3: Quy trình nhận diện thao tác hạ quyền từ đặc tả OpenAPI

tác tạo tài khoản mới hay khóa tài khoản, mang nhãn thông thường.

Hai mô hình ngôn ngữ lớn gán nhãn khởi đầu, mỗi mô hình một lượt độc lập. Sau đó 602 thao tác được soát tay: 66 thao tác hai lượt gán lệch nhau, 165 trong 173 thao tác hai lượt cùng gán nhãn chuyển trạng thái và khớp nhau hoàn toàn, và 371 thao tác hai lượt cùng gán nhãn thông thường, chọn ngẫu nhiên. Mẫu ngẫu nhiên này không có thao tác chuyển trạng thái nào. Thao tác chưa được soát giữ nhãn mà hai lượt gán cùng cho. Tập nhãn cuối có 209 thao tác chuyển trạng thái.

Mô hình dùng trong công cụ được học từ toàn bộ tập này và được chấm trên một tập đặc tả riêng (Mục 5.3). Ngoài bảng trọng số dùng để xếp hạng thao tác, mô

hình còn các bảng trọng số riêng để chọn trường vai trò và trường định danh tài khoản nạn nhân. Giá trị hạ quyền phụ thuộc kiểu của trường vai trò (Bảng 3.5); chỉ trường kiểu liệt kê và kiểu chuỗi dùng mô hình. Bộ dựng loại những ứng viên không đề xuất được giá trị hạ quyền nào, chẳng hạn vì không tìm thấy trường vai trò. Trên pyLoad, lần hạ quyền có hiệu lực nhờ giá trị `role` do pha tìm kiếm để lại; công cụ không tự suy ra giá trị này (Mục 5.4).

Bảng 3.5: Cách chọn giá trị hạ quyền theo kiểu của trường vai trò

Kiểu trường	Giá trị hạ quyền	Ví dụ
Liệt kê	Các giá trị khai báo, xếp theo mức mô hình coi là quyền thấp, trừ vai trò hiện tại của nạn nhân	reports-api: `VIEWER`
Logic	Giá trị đảo	artifact-keeper: `is_admin = false`
Số	Lần lượt `1`, `0`, `2`, thứ tự cố định trong mã	pyLoad: `permission = 1`
Chuỗi	Danh sách giá trị quyền thấp của mô hình	
Mảng	Mảng rỗng	

Với mỗi đặc trưng t, mô hình tính một trọng số logarit theo tần suất của t ở hai lớp. Điểm của một thao tác *op* được tính theo Công thức (3.2):

$$w(t) = \ln \frac{d_T + 1}{N_T + 2} - \ln \frac{d_N + 1}{N_N + 2}, \qquad \text{score}(op) = \ln \frac{N_T}{N_N} + \sum_{t \in op} w(t). \tag{3.2}$$

Trong đó, d_T và d_N là số thao tác thuộc lớp chuyển trạng thái và lớp thông thường có chứa t, còn N_T và N_N là tổng số thao tác của hai lớp. Thao tác có $\text{score}(op) > 0$ được coi là ứng viên chuyển trạng thái. Trường hợp hai thao tác có điểm số bằng nhau, bộ dựng ưu tiên theo thứ tự xuất hiện ban đầu của chúng trong đặc tả OpenAPI. Điểm số chỉ dùng để xếp hạng các ứng viên và chưa được hiệu chuẩn để đọc như một xác suất hậu nghiệm.

Đặc trưng của một thao tác gồm ba nhóm: các từ trong đường dẫn (`p:`), các từ trong tên tham số truy vấn và tên trường của thân yêu cầu (`f:`), và phương thức HTTP (`v:`). Tên được tách thành từ tại các ký tự không phải chữ cái hay chữ số,

và tại chỗ đổi chữ hoa trong tên kiểu camelCase, rồi chuyển về chữ thường; từ toàn chữ số và từ ngắn hơn 4 ký tự bị bỏ. Mô hình chỉ giữ những đặc trưng xuất hiện trong thao tác chuyển trạng thái của ít nhất hai nhà cung cấp API, để mô hình không khớp theo cách đặt tên của riêng một công ty. Đặc trưng được giữ vẫn có thể mang trọng số âm, chẳng hạn `v:POST`. Tên tham số đường dẫn đã có trong `p:`, nên không được lấy lại vào `f:`; nếu lấy lại, cùng một từ được cộng hai lần.

Bảng 3.6 tính điểm của thao tác `PUT /users/{username}/role` trên reports-api. Tiên nghiệm âm vì chỉ 209 trong 10.381 thao tác của tập học mang nhãn chuyển trạng thái. Năm đặc trưng có mặt đưa điểm lên 5,63. Đây là thao tác duy nhất của reports-api có điểm dương. Thao tác `PUT /reports/{id}` chỉ có đặc trưng `v:PUT` trong mô hình: các từ `reports`, `title`, `classification` không có trong mô hình, còn `id` ngắn hơn 4 ký tự. Điểm của thao tác này là $-3{,}885 + 0{,}250 = -3{,}64$.

Bảng 3.6: Điểm của thao tác `PUT /users/{username}/role` trên reports-api

Thành phần	Trọng số	Cộng dồn
Tiên nghiệm	−3,885	−3,885
`p:users`	+2,135	−1,750
`p:username`	+2,449	+0,698
`p:role`	+2,755	+3,453
`f:role`	+1,925	+5,378
`v:PUT`	+0,250	+5,628

Bộ dựng và bộ phán quyết dùng chung quy tắc $\text{score}(op) > 0$. Bộ phán quyết chỉ nhận lời gọi ở giữa làm thao tác chuyển trạng thái khi lời gọi đó thỏa quy tắc, và bỏ qua source thỏa quy tắc. Vì vậy bộ dựng không dựng bộ ba nào mà bộ phán quyết từ chối. Ngưỡng tính cả tiên nghiệm. Nếu bỏ tiên nghiệm, `PUT /reports/{id}` đạt +0,250 nhờ `v:PUT`, bị coi là thao tác chuyển trạng thái và bị loại khỏi vai trò source; Oracle 970 khi đó không bao giờ báo vi phạm trên reports-api.

Giả định độc lập của mô hình bị vi phạm ở các từ thường đi cùng nhau, như `user`, `username`, `email`. Trong Bảng 3.6, hai đặc trưng về người dùng góp 4,58 trong 5,63 điểm. Điểm số vì vậy quá tự tin: điểm 5,63 ứng với hậu nghiệm 0,996, trong khi trên tập chấm chỉ 101 trong 624 thao tác có điểm dương mang nhãn chuyển trạng thái (Mục 5.3).

Chương 4

Cài đặt thực nghiệm và tích hợp hệ thống

Chương này trình bày cách cài đặt Oracle 970 trong EvoMaster, bốn ứng dụng mục tiêu và thiết lập chạy thực nghiệm.

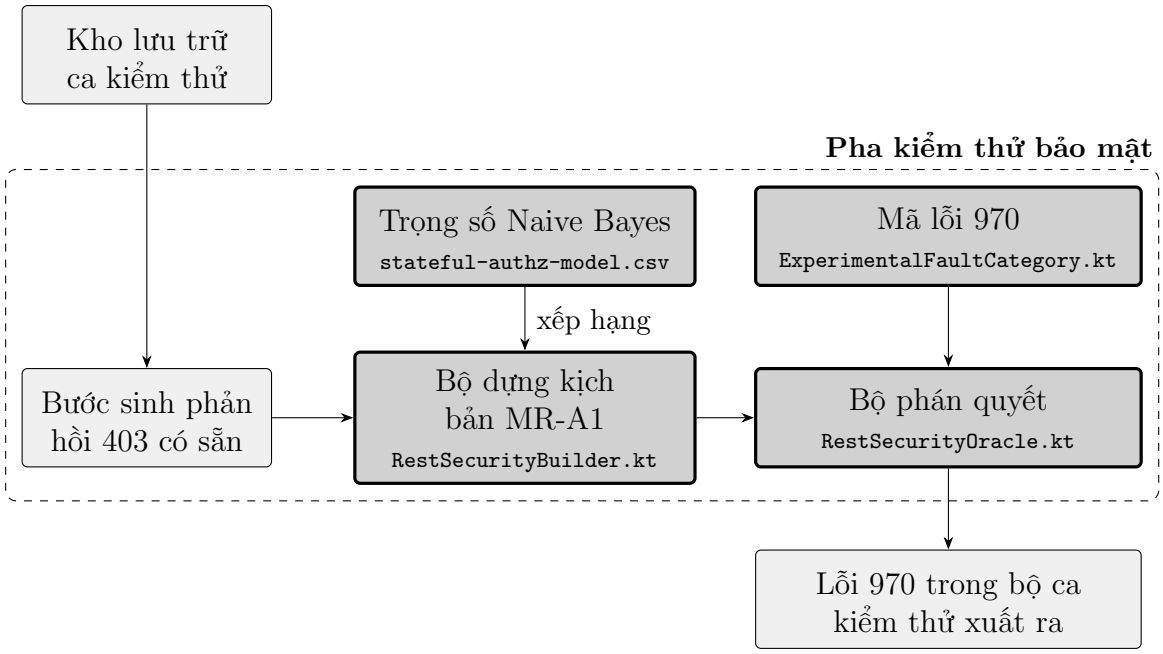
4.1 Tích hợp Oracle 970 vào mã nguồn EvoMaster

Phần mở rộng của Oracle 970 được viết bằng Kotlin (ngôn ngữ của phần core EvoMaster), ngay trong mã nguồn EvoMaster 6.1.1 [9]. Phần mã thêm vào nằm trong ba tệp Kotlin có sẵn của EvoMaster, cùng một tệp trọng số. EvoMaster chạy ở chế độ hộp đen: công cụ đọc đặc tả OpenAPI và gửi yêu cầu HTTP tới ứng dụng mục tiêu đang chạy. Pha tìm kiếm vẫn dùng SMARTS, thuật toán mặc định của chế độ hộp đen (Mục 2.1). Mã nguồn của phần mở rộng nằm trên GitHub tại `https://github.com/minhnguyen-vnu/evomaster-stateful-authz`, nhánh `stateful-authz-oracle`.

Trong EvoMaster, phần mở rộng chạy ở pha kiểm thử bảo mật, sau pha tìm kiếm và pha rút gọn ca kiểm thử (Hình 4.1). Bộ dựng kịch bản kiểm thử đọc các ca kiểm thử trong kho lưu trữ, dùng trọng số của mô hình Naive Bayes để xếp hạng thao tác chuyển trạng thái, rồi dựng kịch bản MR-A1. Trọng số nằm trong một tệp đóng gói cùng công cụ, nên các lượt chạy không phải huấn luyện lại mô hình. Bộ phán quyết đọc mã trạng thái HTTP của bộ ba thao tác và kết luận có vi phạm hay không. Mã lỗi 970 nằm trong nhóm oracle thử nghiệm của EvoMaster, nên Oracle 970 chỉ chạy khi nhóm này được bật.

Các bộ sinh ca kiểm thử trong pha kiểm thử bảo mật dùng chung một quỹ thời gian, tính theo tỷ lệ của thời gian tìm kiếm. Vì bộ sinh xếp sau cùng có thể bị bỏ qua khi quỹ này cạn, bộ dựng kịch bản MR-A1 được đặt ngay sau bước sinh phản hồi 403 có sẵn của EvoMaster. Ở vị trí này, bộ dựng còn chọn được thao tác gốc trong số các thao tác vừa bị từ chối ở bước 403.

Mỗi kịch bản do bộ dựng tạo mang nhãn ca kiểm thử bảo mật, và EvoMaster thực thi kịch bản đó như mọi ca kiểm thử khác. Bộ dựng bật một cờ trước khi thực thi



Hình 4.1: Các thành phần của phần mở rộng trong pha kiểm thử bảo mật

kịch bản của mình và tắt cờ ngay sau đó. Bộ phán quyết chỉ xét ca kiểm thử chạy trong lúc cờ bật; ca kiểm thử của pha tìm kiếm và của các oracle khác không bao giờ bị xét theo MR-A1.

Bộ ba dùng nguyên cơ chế xác thực sẵn có của EvoMaster. EvoMaster đăng nhập mỗi tài khoản một lần ở đầu mỗi ca kiểm thử và dùng token đó cho mọi lời gọi trong ca. Đặt cả bộ ba vào một ca kiểm thử thì follow-up dùng lại token cũ. Bước đăng nhập lại chạy lại chuỗi lời gọi của nạn nhân trong một ca kiểm thử mới, và EvoMaster đăng nhập lại ở đầu ca đó.

Ở chế độ hộp đen, EvoMaster xuất bộ ca kiểm thử dưới dạng mã Python. Hình 4.2 trích ca kiểm thử mà công cụ sinh cho kịch bản ở Bảng 3.3. Lỗi 970 được ghi thành một dòng chú thích ngay trước follow-up, và ca kiểm thử khẳng định follow-up nhận mã 200.

4.2 Ứng dụng mục tiêu

Mỗi ứng dụng mục tiêu cần có thao tác hạ quyền qua REST API, thao tác ghi dành cho quản trị viên và lỗ hổng phân quyền có trạng thái. Thực nghiệm dùng bốn ứng dụng (Bảng 4.1): reports-api và vampi-seeded mang lỗi tự gieo, pyLoad và artifact-keeper là ứng dụng mã nguồn mở mang lỗi thật.

```
def test_0_put_on_report_stillAllowedAfterPermissionRevoked(self):
    ...
    res_0 = requests.post(self.baseUrlOfSut + "/reports", ...)
    ...
    res_1 = requests.put(... "/reports/505" ..., headers=headers, data=body)
    assert res_1.status_code == 200
    ...
    res_2 = requests.put(self.baseUrlOfSut + "/users/alice/role", ...)
    assert res_2.status_code == 200
    # Fault970. Stateful Authorization: privilege persists after a permission-reducing transition.
    ...
    res_3 = requests.put(... "/reports/505" ..., headers=headers, data=body)
    assert res_3.status_code == 200
```

Hình 4.2: Trích ca kiểm thử 970 do công cụ sinh trên reports-api (rút gọn)

Mỗi ứng dụng có hai biến thể: biến thể có lỗ hổng dùng để đo khả năng phát hiện, biến thể đã vá dùng để đo báo động giả. Bản vá của reports-api, vampi-seeded và pyLoad được viết riêng cho khóa luận; bản vá của artifact-keeper là của nhà phát triển. Căn cứ để kỳ vọng ứng dụng từ chối sau khi hạ quyền là bản ghi CVE (pyLoad), bản vá của nhà phát triển (artifact-keeper) và biến thể đã vá (hai ứng dụng mang lỗi tự gieo).

Bảng 4.1: Các ứng dụng mục tiêu

Ứng dụng	**Nền tảng**	**Số thao tác**	**Xác thực**	**Nguồn lỗi**
reports-api	Spring Boot (Java)	9	JWT	Lỗi tự gieo
vampi-seeded	Flask (Python)	16	JWT	Lỗi tự gieo, fork của VAmPI
pyLoad	Flask (Python)	105	Cookie phiên	CVE-2026-41133
artifact-keeper	Rust	389	JWT	Lỗi #1821 (đã vá)

4.2.1 Ứng dụng reports-api

reports-api là dịch vụ REST viết riêng cho thực nghiệm bằng Spring Boot 3 và Java 17. Trên ứng dụng này, kết quả mong đợi của Oracle 970 ở mỗi biến thể đã

biết trước. Máy chủ cấp token JWT khi đăng nhập. Thao tác ghi trên báo cáo và thao tác hạ quyền `PUT /users/{username}/role` đòi vai trò quản trị. Ở biến thể có lỗ hổng, máy chủ đọc vai trò từ claim (trường thông tin mang trong JWT) thay vì tra vai trò hiện tại trong cơ sở dữ liệu. Với token cấp trước khi đổi vai trò, tài khoản đã bị hạ quyền vẫn ghi được dữ liệu. Hai biến thể chạy từ cùng một bản đóng gói và khác nhau ở giá trị cấu hình chọn nguồn đọc vai trò.

4.2.2 Ứng dụng vampi-seeded

vampi-seeded là bản fork (bản sao tách nhánh) của dự án mã nguồn mở VAmPI, dùng để thử Oracle 970 trên mã nguồn và quy ước đặt đường dẫn của một dự án bên thứ ba. VAmPI gốc không có chức năng đổi vai trò. Hai endpoint mới đặt tên theo quy ước đường dẫn của VAmPI: `PUT /users/v1/{username}/role` để hạ quyền và một thao tác ghi cập nhật dữ liệu sách dành cho quản trị viên. Phần gieo lỗi sửa sáu tệp của VAmPI. Lúc đăng nhập, ứng dụng ghi cờ quản trị vào claim của token; một biến môi trường chọn giữa hai biến thể. Thời hạn token được nâng từ 60 lên 3600 giây để token không hết hạn giữa một bộ ba.

4.2.3 Ứng dụng pyLoad

pyLoad là trình quản lý tải viết bằng Python/Flask, mang lỗ hổng CVE-2026-41133 do bên thứ ba công bố độc lập [2]. Bản ghi CVE xếp lỗ hổng vào CWE-613, mục mà MITRE đã ngừng dùng từ CWE 5.0; trong các mục MITRE gợi ý thay thế, CWE-1464 khớp với lỗi này [6, 20]. pyLoad chép vai trò vào phiên lúc đăng nhập. Khi vai trò thay đổi, ứng dụng ghi vai trò mới vào cơ sở dữ liệu nhưng giữ nguyên phiên đã cấp.

Thực nghiệm dùng pyLoad bản dev97, bản cuối cùng mà bản ghi CVE xếp vào diện bị ảnh hưởng. Nhà phát triển vá lỗi từ bản dev98 bằng cách xóa mọi phiên của một người dùng khi quyền của người đó đổi qua endpoint `/json/update_users`, khi đổi mật khẩu hoặc khi xóa tài khoản. Khóa luận không dùng bản vá này làm đối chứng: bản vá thu hồi phiên thay vì từ chối thao tác bằng mã 403. Biến thể đã vá dùng trong thực nghiệm tra lại vai trò hiện tại từ cơ sở dữ liệu ở cả hai chỗ đọc vai trò từ phiên: phần kiểm tra quyền của nhóm endpoint `/json/*` và của nhóm `/api/*`.

So với cấu hình mặc định, môi trường thử nghiệm trên pyLoad có các điều chỉnh sau. Kiểm tra chống giả mạo yêu cầu liên trang (CSRF) bị tắt trên cả hai biến thể, và ứng dụng có thêm một endpoint khôi phục dữ liệu tài khoản. Bản cài đặt mặc định chỉ có một tài khoản, còn MR-A1 cần một tài khoản nạn nhân và một tài khoản quản trị, nên ứng dụng được tạo sẵn thêm một tài khoản quản trị. pyLoad chỉ sinh đặc tả OpenAPI cho nhóm endpoint `/api/*`; đặc tả của nhóm `/json/*` được soạn thủ công. Cấu hình công cụ loại khỏi phạm vi kiểm thử hai endpoint làm dừng tiến trình pyLoad. Tệp chứa toàn bộ logic phân quyền của biến thể có lỗ hổng giữ nguyên mã upstream.

4.2.4 Ứng dụng artifact-keeper

artifact-keeper là máy chủ lưu trữ gói phần mềm (artifact registry) mã nguồn mở, viết bằng Rust [3]. Ở commit có lỗi (`e56d3aa7e2`), máy chủ ghi cờ `is_admin` vào claim của JWT lúc đăng nhập và đọc lại cờ này từ claim mà không tra cơ sở dữ liệu. Quản trị viên đã bị hạ quyền vẫn giữ quyền quản trị tới khi token hết hạn sau 30 phút. Nhà phát triển sửa lỗi này ở commit `e64cc5c608` (issue #1821): token cấp trước lần thay đổi quyền bị từ chối với mã 401. Thao tác hạ quyền là `PATCH /api/v1/users/{id}` với trường `is_admin` trong thân yêu cầu. Cả hai commit được kiểm tra thủ công qua HTTP trước khi chạy công cụ.

Trên artifact-keeper, chỉ cấu hình dựng container và đặc tả OpenAPI được thay đổi. Endpoint `DELETE /users/{id}` được bỏ khỏi đặc tả để công cụ không xóa các tài khoản kiểm thử; mã ứng dụng, lỗi, bản vá và phần còn lại của đặc tả là của nhà phát triển. Mã nguồn của bộ dựng, bộ phán quyết và mô hình Naive Bayes không chứa chuỗi nào riêng của ứng dụng này, và đặc tả của artifact-keeper không thuộc tập ngữ liệu huấn luyện.

4.3 Thiết lập thực nghiệm

Mọi lượt chạy đánh giá đều bật các oracle thử nghiệm của EvoMaster, và dữ liệu của ứng dụng được khởi tạo lại trước mỗi lượt để các lượt độc lập với nhau. EvoMaster chạy trước một ca kiểm thử cho mỗi thao tác với mỗi tài khoản, nên artifact-keeper (389 thao tác) chạy với thời gian tìm kiếm dài hơn và tốc độ gửi yêu cầu cao hơn các ứng dụng còn lại (Bảng 4.2).

Bảng 4.2: Thiết lập chạy theo ứng dụng

Ứng dụng	**Thời gian tìm kiếm**	**Yêu cầu mỗi phút**	**Tỷ lệ thời gian pha bảo mật**	**Seed mỗi biến thể**
reports-api	150 giây	120	0,6	20
vampi-seeded	150 giây	120	0,6	20
pyLoad	10 phút	200	0,6	5
artifact-keeper	20 phút	600	0,25	5 bản lỗi, 3 bản vá

Chương 5

Thực nghiệm và đánh giá kết quả

Chương này đánh giá khả năng phát hiện vi phạm, mức báo động giả và khả năng nhận diện thao tác chuyển trạng thái của Oracle 970. Kết quả được so sánh với baseline, tức EvoMaster 6.1.1 chạy với các oracle phân quyền sẵn có, không kèm Oracle 970.

5.1 Thiết kế thực nghiệm

Để đánh giá khả năng phát hiện vi phạm và mức báo động giả của Oracle 970, thực nghiệm dùng thiết kế cặp đối chứng trên hai biến thể của mỗi ứng dụng mục tiêu. Biến thể có lỗ hổng dùng để đo tỷ lệ phát hiện vi phạm phân quyền có trạng thái. Trên biến thể đã vá, mọi cảnh báo phát sinh đều được tính là báo động giả (false positive, FP). Tổng cộng 98 lượt chạy ở chế độ hộp đen được thực hiện theo thiết lập ở Mục 4.3, gồm 50 lượt trên biến thể có lỗ hổng và 48 lượt trên biến thể đã vá. Mọi lượt dùng cùng một bản đóng gói của công cụ.

Mỗi lượt chạy ghi lại mã trạng thái HTTP của các bộ ba đã thực thi, nên số lượt dựng được bộ ba và số lượt báo vi phạm được thống kê riêng. Ngoài 98 lượt chạy trên các ứng dụng mục tiêu, khả năng xếp hạng thao tác chuyển trạng thái được đánh giá ngoại tuyến trên một tập đặc tả OpenAPI riêng (Mục 5.3).

5.2 Hiệu quả phát hiện và độ chính xác

Bảng 5.1 tổng hợp số lượt dựng được bộ ba và số lượt báo vi phạm trên từng biến thể. Cột cuối của bảng ghi khoảng tin cậy Clopper–Pearson 95% của tỷ lệ phát hiện trên biến thể có lỗ hổng [21]. Khoảng này tính chính xác từ phân phối nhị thức, nên xác suất khoảng chứa tỷ lệ thật đạt ít nhất 95% kể cả với mẫu 5 lượt.

Tính chung bốn ứng dụng, Oracle 970 phát hiện vi phạm ở 34/50 lượt trên biến thể có lỗ hổng và không báo vi phạm ở lượt nào trong 48 lượt trên biến thể đã vá. Kiểm định chính xác Fisher một phía cho thấy tỷ lệ báo vi phạm khác nhau

Bảng 5.1: Kết quả phát hiện và báo động giả theo lượt chạy

Ứng dụng	Biến thể	Lượt có bộ ba	Lượt báo vi phạm	Khoảng tin cậy 95% (%)
reports-api	có lỗ hổng	19/20	19/20	[75,1; 99,9]
reports-api	đã vá	19/20	0/20	—
vampi-seeded	có lỗ hổng	6/20	6/20	[11,9; 54,3]
vampi-seeded	đã vá	6/20	0/20	—
pyLoad	có lỗ hổng	5/5	5/5	[47,8; 100]
pyLoad	đã vá	5/5	0/5	—
artifact-keeper	có lỗ hổng	4/5	4/5	[28,4; 99,5]
artifact-keeper	đã vá	0/3	0/3	—

giữa hai biến thể ở reports-api ($p < 0{,}001$), vampi-seeded ($p \approx 0{,}010$) và pyLoad ($p \approx 0{,}0040$) [22]. Với artifact-keeper, $p \approx 0{,}071$: 4/5 lượt so với 0/3 lượt chưa đủ để kết luận ở mức ý nghĩa 0,05.

Bảng 5.2 phân loại lượt chạy theo nhãn TP (phát hiện đúng), FN (bỏ sót), FP (báo động giả) và TN (âm tính thật). Trên biến thể đã vá, lượt không dựng được bộ ba không có phán quyết nào, nên được đếm ở cột riêng và không tính là TN.

Bảng 5.2: Phân loại kết quả theo lượt chạy

	Biến thể có lỗ hổng		Biến thể đã vá		
Ứng dụng	TP	FN	FP	TN	Không dựng được bộ ba
reports-api	19	1	0	19	1
vampi-seeded	6	14	0	6	14
pyLoad	5	0	0	5	0
artifact-keeper	4	1	0	0	3
Tổng	34	16	0	30	18

Xét riêng các lượt dựng được bộ ba, Oracle 970 báo vi phạm ở cả 34 lượt trên biến thể có lỗ hổng và không phát sinh cảnh báo nào trong 30 lượt trên biến thể đã vá. Khoảng tin cậy Clopper–Pearson 95% của tỷ lệ báo động giả này là [0; 11,6]%. Mỗi

lượt dựng được bộ ba có đúng một bộ ba, với chuỗi mã trạng thái `200/200/200` trên biến thể có lỗ hổng và `200/200/403` trên biến thể đã vá. Tỷ lệ phát hiện tính trên các lượt có bộ ba gần như là hệ quả của định nghĩa, vì tiêu chí hợp lệ của bộ ba dùng chính mã follow-up mà bộ phán quyết đọc. Phép đo độc lập là số báo động giả trên biến thể đã vá.

Trên bản vá của artifact-keeper, cả 3 lượt đều không dựng được bộ ba: thao tác hạ quyền thành công nhưng token cũ nhận mã 401 ở bước follow-up, mà mã 401 nằm ngoài phán quyết (Bảng 3.1), nên bộ ba bị hủy. Ba lượt này không có báo động giả, nhưng cũng không được tính là âm tính thật.

Toàn bộ 16 lượt bỏ sót đều không dựng được bộ ba hợp lệ để kích hoạt bộ phán quyết. Nguyên nhân của các lượt này được phân tích tại Mục 5.5.

5.3 Hiệu quả xếp hạng thao tác chuyển trạng thái

Mô hình Naive Bayes đi kèm công cụ được chấm trên 237 bản đặc tả không tham gia huấn luyện, gồm 8.674 thao tác ghi. Trong 153 thao tác chuyển trạng thái của tập này, 101 thao tác (66,0%; khoảng Clopper–Pearson 95% là [57,9; 73,5]) có điểm dương. Trong 624 thao tác có điểm dương, 523 thao tác (83,8%) mang nhãn thông thường, nên điểm dương chỉ đủ để chọn ứng viên, chưa đủ để kết luận một thao tác hạ quyền. Các tỷ lệ này tính theo nhãn hiện có: 8.130 thao tác mà hai lần gán cùng gọi thông thường chưa được soát tay, nên trong đó vẫn có thể còn thao tác chuyển trạng thái bị sót. Nhãn của tập này được gán theo quy trình tương tự tập học (Mục 3.4). Sau khi chấm, 44 thao tác mà hai lần gán bất đồng được soát lại, và 4 thao tác đổi sang nhãn chuyển trạng thái; trước lần soát này, tỷ lệ là 100/149 (67,1%).

Bảng 5.3 cho biết vị trí của thao tác chuyển trạng thái thật trên đặc tả của từng ứng dụng mục tiêu. Cột *dùng được* đếm các thao tác có điểm dương mà bộ dựng đề xuất được giá trị hạ quyền. Mô hình đặt thao tác thật ở hạng 1 trên reports-api, vampi-seeded và pyLoad. Trên artifact-keeper, thao tác này đứng hạng 3 trong 4 ứng viên; bộ dựng thử lần lượt các ứng viên và dựng được bộ ba trên ứng viên thứ ba ở 4 trong 5 lượt chạy trên biến thể có lỗ hổng.

Bảng 5.3: Hạng của thao tác chuyển trạng thái thật trên các ứng dụng mục tiêu

Ứng dụng	Thao tác chuyển trạng thái	Thao tác ghi	Điểm dương	Dùng được	Hạng
reports-api	`PUT /users/{username}/role`	6	1	1	1
vampi-seeded	`PUT /users/v1/{username}/role`	8	5	1	1
pyLoad	`POST /api/set_user_permission`	62	4	3	1
artifact-keeper	`PATCH /api/v1/users/{id}`	216	14	4	3

5.4 Nghiên cứu tình huống trên lỗi thật

pyLoad và artifact-keeper là hai ứng dụng của bên thứ ba: lỗi và mã ứng dụng do nhà phát triển của chúng viết. Đặc tả của artifact-keeper là của nhà phát triển, trừ một endpoint bị bỏ; trên pyLoad, đặc tả nhóm `/json/*` được soạn thủ công cho khóa luận (Mục 4.2.3).

Trên pyLoad, công cụ tự nhận diện thao tác chuyển trạng thái `POST /api/set_user_permission` từ đặc tả, xếp hạng 1 trong 3 ứng viên dùng được. Ở cả 10 lượt chạy, bộ ba dùng source `POST /json/status`, nạn nhân là tài khoản `pyload` và tài khoản quản trị là `alice`; mỗi lượt cần một lần khôi phục quyền cho tài khoản quản trị trước khi dựng được bộ ba. Oracle 970 báo vi phạm ở 5/5 lượt trên biến thể có lỗ hổng và 0/5 lượt trên biến thể đã vá. Thao tác hạ quyền gửi `permission=1` cùng một giá trị `role` do pha tìm kiếm để lại, luôn khác 0, giá trị mà pyLoad dành cho quản trị viên. Công cụ không tự suy ra giá trị `role` này; bước đăng nhập lại xác nhận quyền đã mất ở từng lượt. Lỗ hổng trên pyLoad còn được kiểm tra thủ công qua HTTP trên cả hai biến thể.

Với artifact-keeper, công cụ tự chọn thao tác `PATCH /api/v1/users/{id}` và đặt trường `is_admin` thành `false`. Ở 4 trong 5 lượt trên commit có lỗi, bộ ba dùng source `POST /api/v1/admin/analytics/snapshot`, một thao tác chỉ dành cho quản trị viên, với chuỗi mã `200/200/200`, và Oracle 970 báo vi phạm. Ở lượt còn lại, bộ dựng dùng source `PUT /api/v1/admin/sso/ldap/{id}`; mọi lần thử trên source này bị hủy hoặc bị bước đăng nhập lại loại, nên lượt đó không dựng được bộ ba. Bản vá của nhà phát triển trả mã 401 cho token cũ, nên không lượt nào dựng được bộ ba (Mục 5.2). Với cùng công cụ và cùng cấu hình chạy, mã nguồn của nhà phát triển trước và sau khi sửa lỗi cho 4/5 và 0/3 lượt báo vi phạm; với

số lượt này, chênh lệch chưa đạt mức ý nghĩa 0,05 ($p \approx 0{,}071$). Khác biệt ở đây đến từ việc bộ ba trên bản vá bị hủy vì mã 401; bản vá không được phán quyết bằng mã 403.

Trên artifact-keeper, bước đăng nhập lại đã loại các bộ ba dựng trên thao tác không hạ quyền. Ở cả 8 lượt chạy, ứng viên thứ hai `POST /api/v1/admin/sso/oidc` cho chuỗi mã `200/200/200`. Token mới của nạn nhân vẫn được chấp nhận, tức thao tác này không hạ quyền nạn nhân, và bước đăng nhập lại loại lần thử đó. Ở lượt bản lỗi không dựng được bộ ba, bước này loại thêm một lần thử trên ứng viên thứ tư `POST /api/v1/users`, cũng với chuỗi `200/200/200`. Tổng cộng bước đăng nhập lại loại 6 lần thử trên bản lỗi và 3 lần thử trên bản vá. Chuỗi `200/200/200` đúng là mẫu mà bộ phán quyết báo vi phạm; khóa luận không chạy cấu hình bỏ bước đăng nhập lại. Trong 4 lượt trên bản lỗi có báo vi phạm, mọi vi phạm đều nằm ở ứng viên thứ ba, thao tác thật sự hạ quyền. Bước đăng nhập lại không loại lần thử nào trên ba ứng dụng còn lại.

5.5 Phân tích nguyên nhân bỏ sót và giới hạn

Toàn bộ 16 lượt bỏ sót trên biến thể có lỗ hổng được chia thành hai nhóm. Nhóm thứ nhất gồm 10 lượt (1 trên reports-api, 8 trên vampi-seeded và 1 trên artifact-keeper), trong đó bộ dựng đã thử dựng bộ ba nhưng mọi lần thử đều bị hủy theo các điều kiện ở Mục 3.3.3 hoặc bị bước đăng nhập lại loại. Ở nhóm thứ hai, gồm 6 lượt còn lại trên vampi-seeded, bộ dựng không thực hiện lần thử nào; nhật ký ở mức mặc định của EvoMaster không ghi lý do cụ thể.

Ở 8 lượt bỏ sót thuộc nhóm thứ nhất trên vampi-seeded, bộ dựng có tổng cộng 48 lần thử bị hủy. Trong đó, 37 lần hủy vì source nhận mã 401, tức token của nạn nhân không được chấp nhận, và 11 lần hủy vì thao tác chuyển trạng thái nhận mã 401, tức token của tài khoản quản trị không được chấp nhận. Trên VAmPI, mã 401 cho biết token không hợp lệ. Điều này khớp với việc các ca kiểm thử của pha tìm kiếm xóa chính các tài khoản kiểm thử, trình bày ở đoạn dưới. Lượt bỏ sót trên reports-api có một lần thử không ghi được mã trạng thái nào. Lượt bỏ sót trên artifact-keeper có 5 lần thử bị hủy vì thao tác chuyển trạng thái nhận mã 400 hoặc 403, và 2 lần thử bị bước đăng nhập lại loại.

Pha tìm kiếm có thể đổi trạng thái ứng dụng trước khi bộ dựng chạy và làm giảm

số bộ ba dựng được. Chẳng hạn, tài khoản nạn nhân có thể bị hạ quyền ngay trong pha này, làm source nhận mã 403, hoặc bị xóa, làm bước khôi phục quyền mất tác dụng. Trên vampi-seeded, các ca kiểm thử do pha tìm kiếm sinh ra gọi thao tác xóa tài khoản `DELETE /users/v1/{username}` nhiều lần trên chính các tài khoản kiểm thử. Các tác dụng phụ này làm giảm số lượt phát hiện; trên các bộ ba hợp lệ đã quan sát, chúng chưa gây ra phán quyết sai nào.

Về mặt thống kê, quy mô mẫu còn nhỏ: 20 seed cho mỗi biến thể của reports-api và vampi-seeded, 5 seed cho mỗi biến thể của pyLoad, 5 seed cho biến thể có lỗi và 3 seed cho bản vá của artifact-keeper. Kiểm định Fisher giả định hai mẫu độc lập, trong khi hai biến thể dùng chung seed, nên các giá trị p chủ yếu mang ý nghĩa tham khảo.

Phán quyết dựa trên mã trạng thái HTTP có giới hạn riêng: mã `2xx` chưa chứng minh trạng thái dữ liệu nghiệp vụ sau thao tác, còn mã 403 không nêu nguyên nhân từ chối. Trên reports-api, 17 trong 19 ca kiểm thử báo 970 có lời gọi tạo báo cáo đứng trước source. EvoMaster dừng chuỗi ở lời gọi tạo tài nguyên đầu tiên thất bại, nên ở các ca này, mã 403 của bước đăng nhập lại đến từ lời gọi tạo báo cáo, vốn cũng chỉ dành cho quản trị viên. Mã này vẫn cho thấy token mới đã mất quyền ghi, dù lời gọi bị từ chối khác thao tác đang xét. Ngoài ra, mã từ chối được cố định là 403 và chưa cấu hình được. Bản vá của artifact-keeper từ chối token cũ bằng mã 401, nên công cụ không dựng được bộ ba nào trên bản vá này.

Nhãn dùng để học và chấm mô hình Naive Bayes do hai mô hình ngôn ngữ lớn gán và chỉ được soát tay một phần. Trong 10.381 thao tác ghi của tập huấn luyện, 9.771 thao tác mà hai lần gán cùng gọi thông thường chưa được soát tay.

Khả năng khái quát hóa bị giới hạn ở một lớp lỗ hổng phân quyền có trạng thái và một quan hệ MR-A1. Ba ứng dụng reports-api, vampi-seeded và artifact-keeper cùng đọc quyền cũ từ claim của JWT; ở artifact-keeper, đó là mã của nhà phát triển. pyLoad lưu vai trò trong phiên. Đường dẫn thao tác hạ quyền trên reports-api và vampi-seeded được đặt khi xây dựng ứng dụng và gieo lỗi, nên kết quả nhận diện trên hai ứng dụng này chưa đại diện cho đặc tả bên thứ ba. Tỷ lệ phát hiện 5/5 trên pyLoad đạt được trong môi trường thử nghiệm có các điều chỉnh tại Mục 4.2.3 và với biến thể đã vá viết riêng cho khóa luận, nên chưa áp dụng trực tiếp cho bản cài đặt mặc định của pyLoad.

5.6 Đối chứng với baseline và kiểm tra upstream

Phép đối chứng kiểm tra baseline và nhánh chính EvoMaster upstream có dựng và phán quyết được chuỗi MR-A1 hay không. Các oracle phân quyền của baseline thuộc nhóm oracle so sánh chéo tài khoản của Şahin và cộng sự, chỉ đối chiếu quyền giữa các tài khoản tại một thời điểm. Chuỗi hạ quyền rồi dùng lại token cũ theo MR-A1 vì thế nằm ngoài các kịch bản mà baseline phán quyết.

Toàn bộ 20 lượt chạy baseline trên biến thể có lỗ hổng của vampi-seeded đều không sinh được bộ ba thao tác dùng lại token cũ. Ở 17/20 lượt, baseline ghi nhận lỗi lộ dấu vết ngăn xếp (stack trace) trong phản hồi HTTP. Pha kiểm thử bảo mật của baseline vì vậy vẫn chạy trong các lượt đó, nên việc thiếu bộ ba không do pha này bị bỏ qua.

Nhánh chính của EvoMaster upstream (commit `ab983ba338`) được chạy thêm với 20 seed trên biến thể có lỗ hổng của reports-api, qua một proxy ghi lại toàn bộ lưu lượng HTTP. Không lượt nào báo lỗi thuộc lớp MR-A1. Trong lưu lượng có 4 lời gọi dùng token cấp lúc tài khoản còn quyền quản trị, gửi sau khi tài khoản đã bị hạ quyền, và nhận mã 200. Không lời gọi nào trong số đó có source thành công bằng cùng token trước lần hạ quyền, không lời gọi nào bị gắn lỗi, và bộ ca kiểm thử xuất ra không giữ chuỗi nào như vậy.

Chương 6

Kết luận và hướng phát triển

Oracle 970 phát hiện được vi phạm MR-A1 trên cả bốn ứng dụng đã khảo sát, nhưng còn giới hạn ở khâu dựng bộ ba và khả năng mở rộng sang ứng dụng khác. Chương này tổng kết các đóng góp, nêu hạn chế và đề xuất hướng phát triển.

6.1 Tổng kết kết quả nghiên cứu

Khóa luận đã mở rộng EvoMaster bằng Oracle 970 để tự động kiểm tra lỗ hổng phân quyền có trạng thái thuộc quan hệ MR-A1. Phần mở rộng chạy trong pha kiểm thử bảo mật và không thay đổi thuật toán tìm kiếm. Các kết quả dưới đây được trình bày theo bốn đóng góp nêu ở Mục 1.4.

Thứ nhất, quan hệ MR-A1 được cài đặt thông qua điều kiện tiền đề và bộ ba thao tác source, transition và follow-up. Mọi lượt bỏ sót trong thực nghiệm đều là lượt không dựng được bộ ba hợp lệ để phán quyết.

Thứ hai, mô hình Naive Bayes tự động nhận diện thao tác chuyển trạng thái từ đặc tả OpenAPI. Trên 237 bản đặc tả không tham gia huấn luyện, 101 trong 153 thao tác chuyển trạng thái có điểm dương; 523 trong 624 thao tác có điểm dương mang nhãn thông thường, nên điểm dương chỉ dùng để chọn ứng viên. Trên pyLoad và artifact-keeper, công cụ tự chọn thao tác hạ quyền từ đặc tả mà không cần khai báo thủ công.

Thứ ba, bước đăng nhập lại loại các bộ ba mà token mới vẫn được chấp nhận, nhằm tránh phán quyết trên những thao tác chuyển trạng thái chưa xác nhận là hạ quyền. Trên artifact-keeper, bước này loại 9 lần thử dựng trên hai thao tác không hạ quyền, dù chuỗi mã của chúng là `200/200/200`.

Thứ tư, thực nghiệm gồm 98 lượt chạy trên bốn ứng dụng. Oracle 970 báo vi phạm ở 19/20 lượt với reports-api, 6/20 lượt với vampi-seeded, 5/5 lượt với pyLoad và 4/5 lượt với artifact-keeper. Trong 48 lượt trên biến thể đã vá, công cụ dựng được bộ ba ở 30 lượt và không báo động giả ở lượt nào. Lỗi trên pyLoad (CVE-2026-41133) và artifact-keeper nằm trong mã của nhà phát triển ứng dụng; trên biến thể đã vá

của pyLoad, Oracle 970 báo vi phạm ở 0/5 lượt. Bản vá của nhà phát triển từ chối token cũ bằng mã 401, nên trong 3 lượt chạy trên bản vá này, công cụ không dựng được bộ ba nào và cũng không báo động giả.

6.2 Hạn chế hiện tại của đề tài

Khóa luận chỉ kiểm chứng quan hệ MR-A1, tức trường hợp hạ vai trò; các quan hệ khác của cùng lớp lỗi được nêu ở Mục 6.3. Mã từ chối được cố định là 403 và chưa cấu hình được, nên bản vá trả 401 như của artifact-keeper không được phán quyết. Các giới hạn còn lại, gồm quy mô mẫu nhỏ và phán quyết dựa trên mã trạng thái HTTP, được phân tích ở Mục 5.5.

6.3 Hướng phát triển trong tương lai

Để nhận diện thao tác chuyển trạng thái trong các đặc tả đặt tên không theo quy ước, có thể dùng mô hình ngôn ngữ lớn (Large Language Model, LLM) đề xuất ứng viên. Naive Bayes hiện chỉ dựa vào các từ trong đường dẫn, tên trường và phương thức. LLM có thể khai thác mô tả trong đặc tả OpenAPI và ngữ cảnh thao tác để đề xuất ứng viên, trong khi phép phán quyết giữ nguyên.

Phần mở rộng cũng có thể nhận thêm các tín hiệu từ chối ngoài mã 403, như mã 401 khi phiên bị vô hiệu hóa, để đánh giá được những bản vá như của artifact-keeper. Ngoài ra, bộ dựng có thể chủ động tạo tài nguyên và tài khoản cần thiết trước khi chạy kịch bản kiểm thử MR-A1, thay vì chỉ dựa vào kho lưu trữ. Cải tiến này có thể giúp 6 trong 16 lượt bỏ sót, nơi bộ dựng không thực hiện lần thử nào; nguyên nhân của 6 lượt này chưa được xác định (Mục 5.5).

Ngoài MR-A1, nghiên cứu tiếp theo có thể xét các quan hệ khác của lớp phân quyền có trạng thái, như chuyển quyền sở hữu tài nguyên và quyền hết hạn theo thời gian. Mỗi quan hệ cần điều kiện tiền đề và cơ chế phán quyết riêng.

Tài liệu tham khảo

[1] OWASP Foundation. OWASP Top 10 API Security Risks – 2023. https://owasp.org/API-Security/editions/2023/en/0x11-t10/, 2023.

[2] MITRE CVE Program. CVE-2026-41133: pyLoad has Stale Session Privilege After Role/Permission Change (Privilege Revocation Bypass). https://cve.org/CVERecord?id=CVE-2026-41133, 2026.

[3] artifact-keeper Contributors. fix(security): invalidate tokens on admin/role privilege change (Closes #1821). https://github.com/artifact-keeper/artifact-keeper/commit/e64cc5c608df4afdd0b35feae5f140a9d809a8f3, 2026.

[4] Andrea Arcuri. RESTful API Automated Test Case Generation with EvoMaster. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 28(1):3:1–3:37, 2019.

[5] Ömür Şahin, Man Zhang, and Andrea Arcuri. Enhancing REST API Fuzzing with Access Policy Violation Checks and Injection Attacks. *arXiv preprint arXiv:2604.00702*, 2026.

[6] MITRE. CWE-1464: Improper Session Invalidation. https://cwe.mitre.org/data/definitions/1464.html, 2026.

[7] Andrea Arcuri, Man Zhang, Susruthan Seran, Juan Pablo Galeotti, Amid Golmohammadi, Onur Duman, Agustina Aldasoro, and Hernan Ghianni. Tool report: EvoMaster—black and white box search-based fuzzing for REST, GraphQL and RPC APIs. *Automated Software Engineering*, 32(1):4, 2025.

[8] Andrea Arcuri. Many Independent Objective (MIO) Algorithm for Test Suite Generation. In *Search Based Software Engineering*, pages 3–17. Springer, 2017.

[9] EvoMaster Contributors. EvoMaster v6.1.1, source code. https://github.com/WebFuzzing/EvoMaster/tree/v6.1.1, 2026.

[10] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering*, 41(5):507–525, 2015.

[11] Tsong Yueh Chen, Shing Chi Cheung, and Shiu Ming Yiu. Metamorphic Testing: A New Approach for Generating Next Test Cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology, 1998.

[12] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Computing Surveys (CSUR)*, 51(1):4:1–4:27, 2018.

[13] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, RFC Editor, May 2015.

[14] BSR INCITS 359 Committee. Role Based Access Control (Draft). Technical report, InterNational Committee for Information Technology Standards (INCITS), April 2003.

[15] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. RESTler: Stateful REST API Fuzzing. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 748–758, 2019.

[16] Andrea Arcuri, Ömür Şahin, and Man Zhang. Fuzzing for Detecting Access Policy Violations in REST APIs. In *Proceedings of the 36th IEEE International Symposium on Software Reliability Engineering (ISSRE)*, pages 130–141, 2025.

[17] Fengyu Liu, Yuan Zhang, Enhao Li, Wei Meng, Youkun Shi, Qianheng Wang, Chenlin Wang, Zihan Lin, and Min Yang. BACScan: Automatic Black-Box Detection of Broken-Access-Control Vulnerabilities in Web Applications. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1320–1333, 2025.

[18] Omar Tsai, Jianing Li, Tsz Tung Cheung, Lejing Huang, Hao Zhu, Jianrui Xiao, Iman Sharafaldin, and Mohammad A. Tayebi. GraphQLer: Enhancing GraphQL Security with Context-Aware API Testing. *arXiv preprint arXiv:2504.13358v2*, 2026.

[19] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP Semantics. RFC 9110, RFC Editor, June 2022.

[20] MITRE. CWE-613: DEPRECATED: Insufficient Session Expiration. `https://cwe.mitre.org/data/definitions/613.html`, 2026.

[21] C. J. Clopper and E. S. Pearson. The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. *Biometrika*, 26(4):404–413, 1934.

[22] Ronald A. Fisher. On the Interpretation of χ^2 from Contingency Tables, and the Calculation of P. *Journal of the Royal Statistical Society*, 85(1):87–94, 1922.

Phụ lục A

Danh mục endpoint của các ứng dụng mục tiêu

Bảng A.1 liệt kê các endpoint tham gia bộ ba do công cụ dựng trên bốn ứng dụng mục tiêu mô tả ở Mục 4.2.

Bảng A.1: Các endpoint trong bộ ba do công cụ dựng trên bốn ứng dụng mục tiêu

Endpoint	**Quyền**	**Vai trò trong kịch bản**
reports-api		
`POST /auth/login`	Mọi tài khoản	Đăng nhập, cấp JWT chứa vai trò
`PUT /reports/{id}`	Quản trị	Source và follow-up
`PUT /users/{u}/role`	Quản trị	Chuyển trạng thái
vampi-seeded		
`POST /users/v1/login`	Mọi tài khoản	Đăng nhập, cấp JWT chứa cờ quản trị
`PUT /books/v1/{t}/secret`	Quản trị	Source và follow-up
`PUT /users/v1/{u}/role`	Quản trị	Chuyển trạng thái
pyLoad		
`POST /login`	Mọi tài khoản	Đăng nhập, tạo cookie phiên
`POST /json/status`	Quyền `LIST`	Source và follow-up
`POST /api/set_user_permission`	Quản trị	Chuyển trạng thái
artifact-keeper		
`POST /api/v1/auth/login`	Mọi tài khoản	Đăng nhập, cấp JWT chứa cờ `is_admin`
`POST /api/v1/admin/analytics/snapshot`	Quản trị	Source và follow-up
`PATCH /api/v1/users/{id}`	Quản trị	Chuyển trạng thái